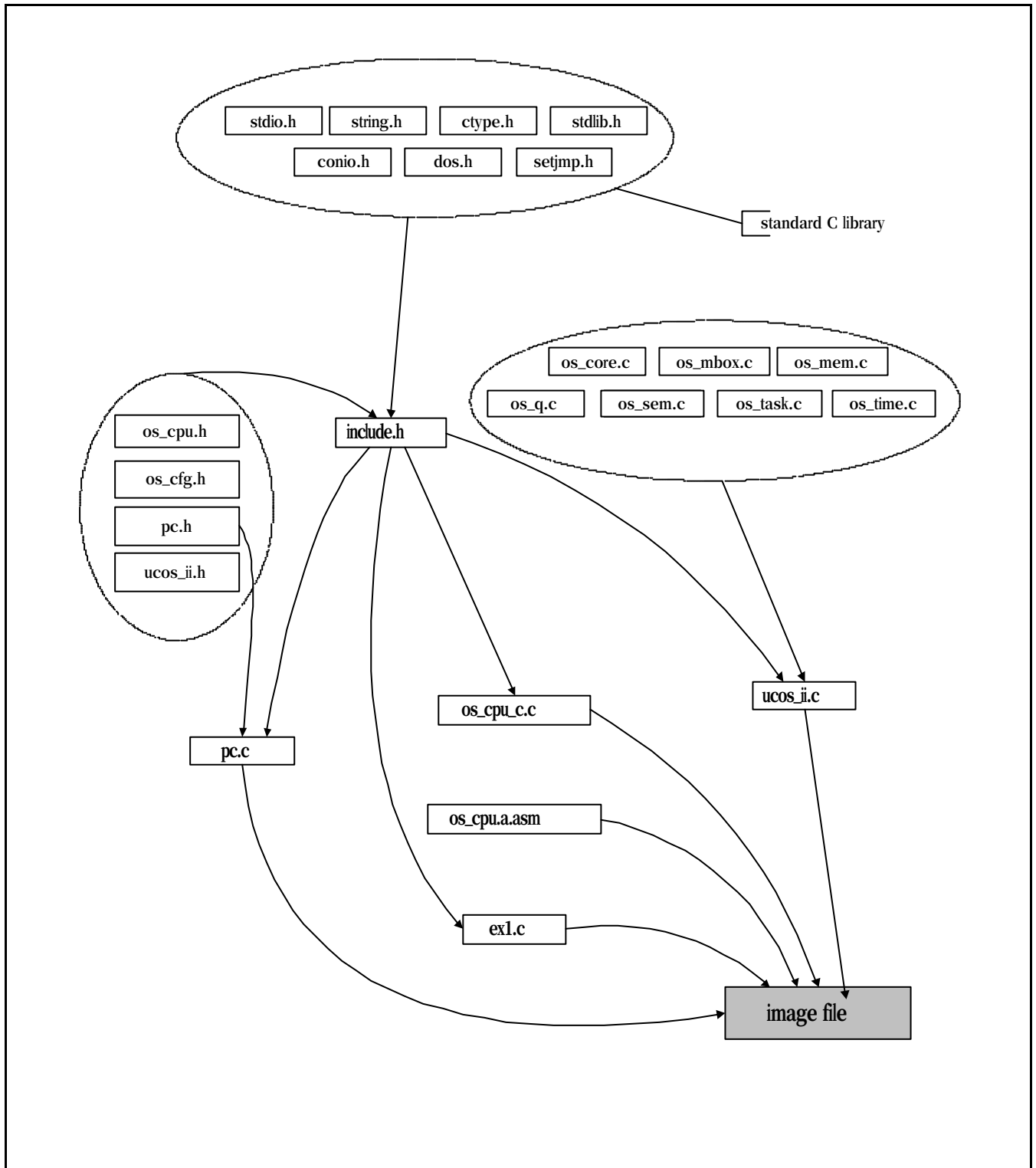
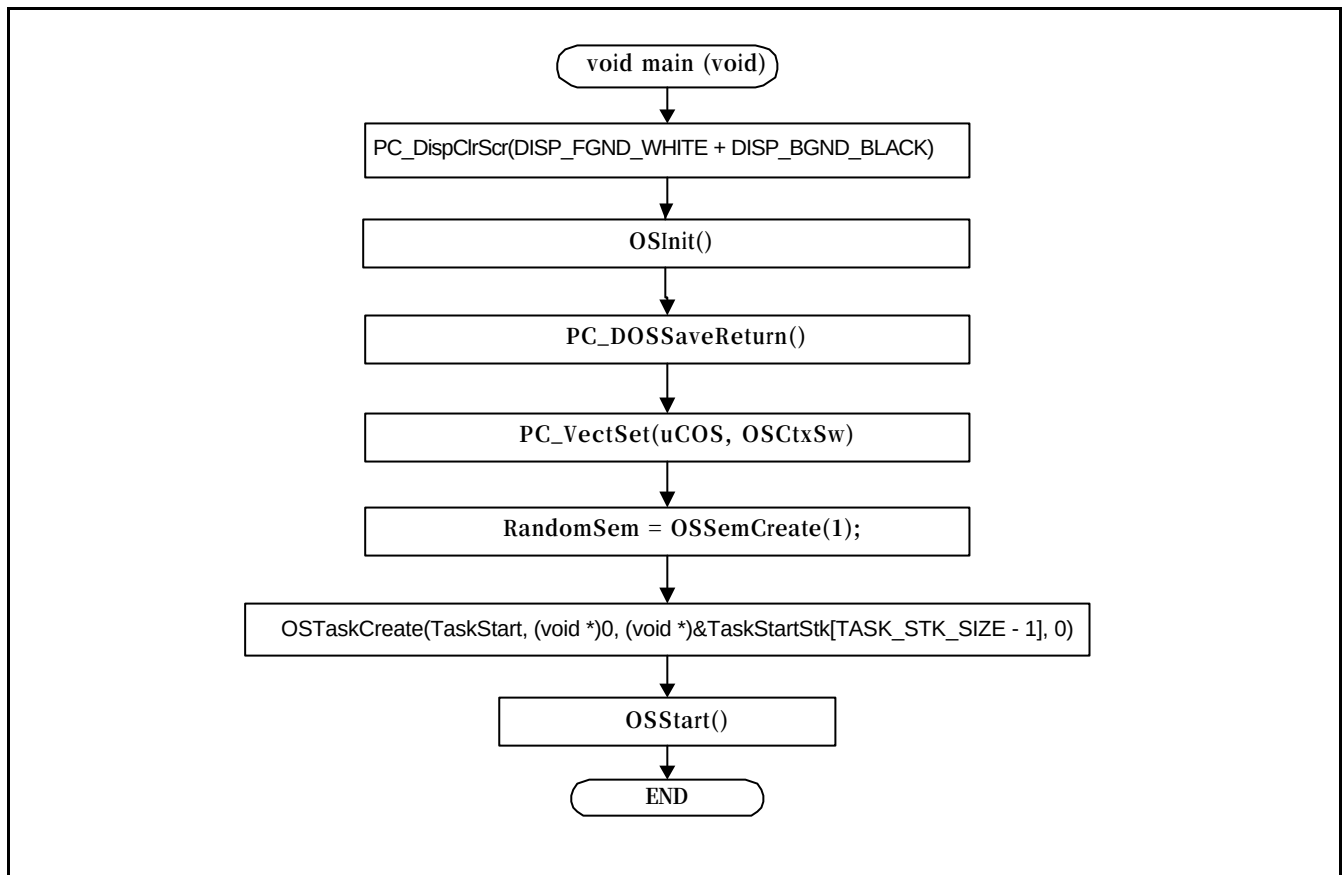


1 Source dependency



< Figure 1-1 uC/OS-II Source dependency >



< Figure 1-2 Ex1l.c main() Funtion >

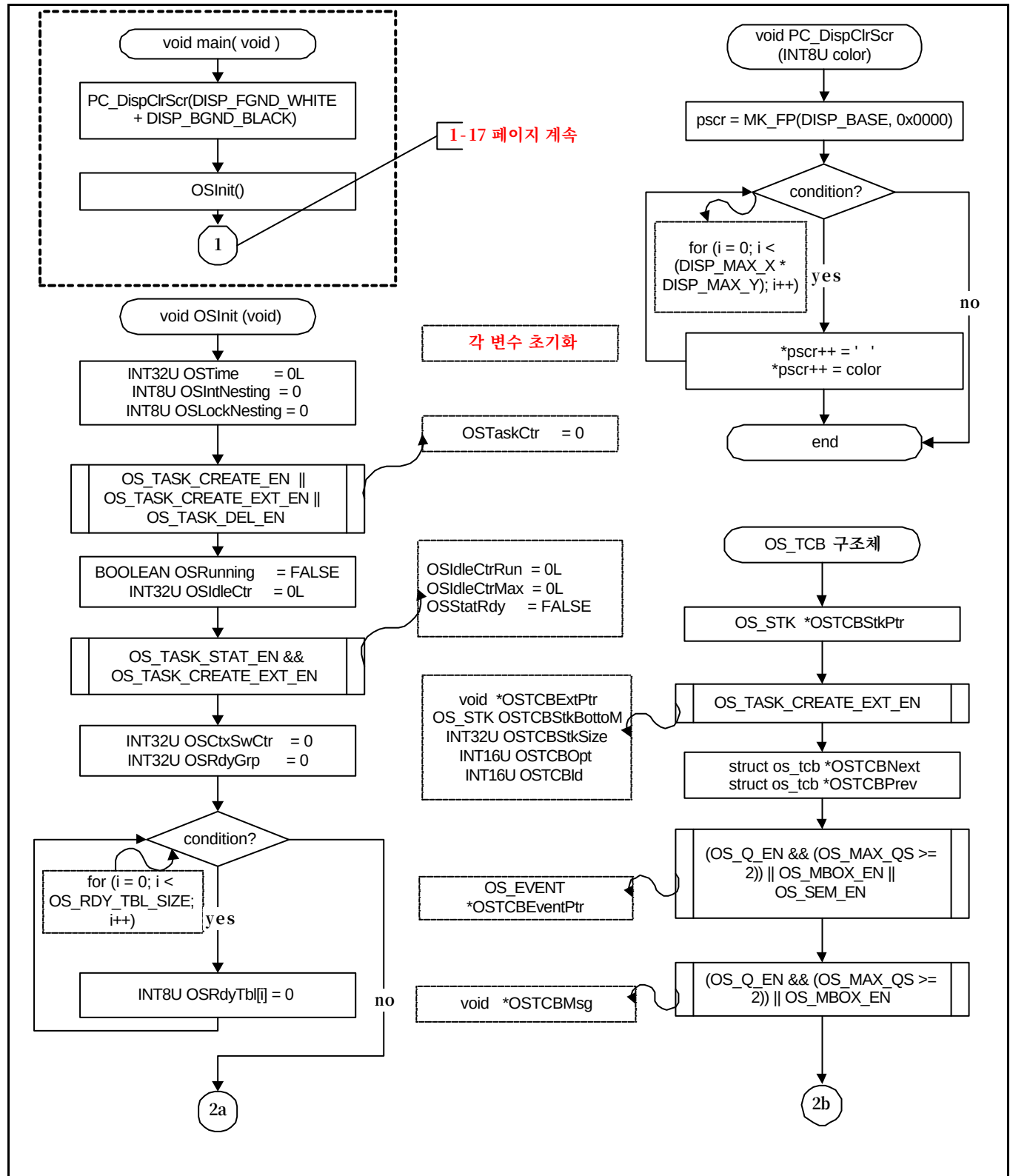
uC의 소스파일에는 x86에 맞게 이미 포팅 된 총 3가지의 예제가 있으며, 그 중 처음 다룰 예제는 ex1l.c 함수이다. 각 예제는 자신만의 os_cfg.h 헤더파일을 가지고 있으며 이 헤더파일내의 여러 변수의 설정에 의해 uC 컴파일시에 특정기능을 추가 또는 제외시킬 수 있다. 아래는 ex1l.c에 관련된 os_cfg.h파일의 내용을 나타낸 것이다.

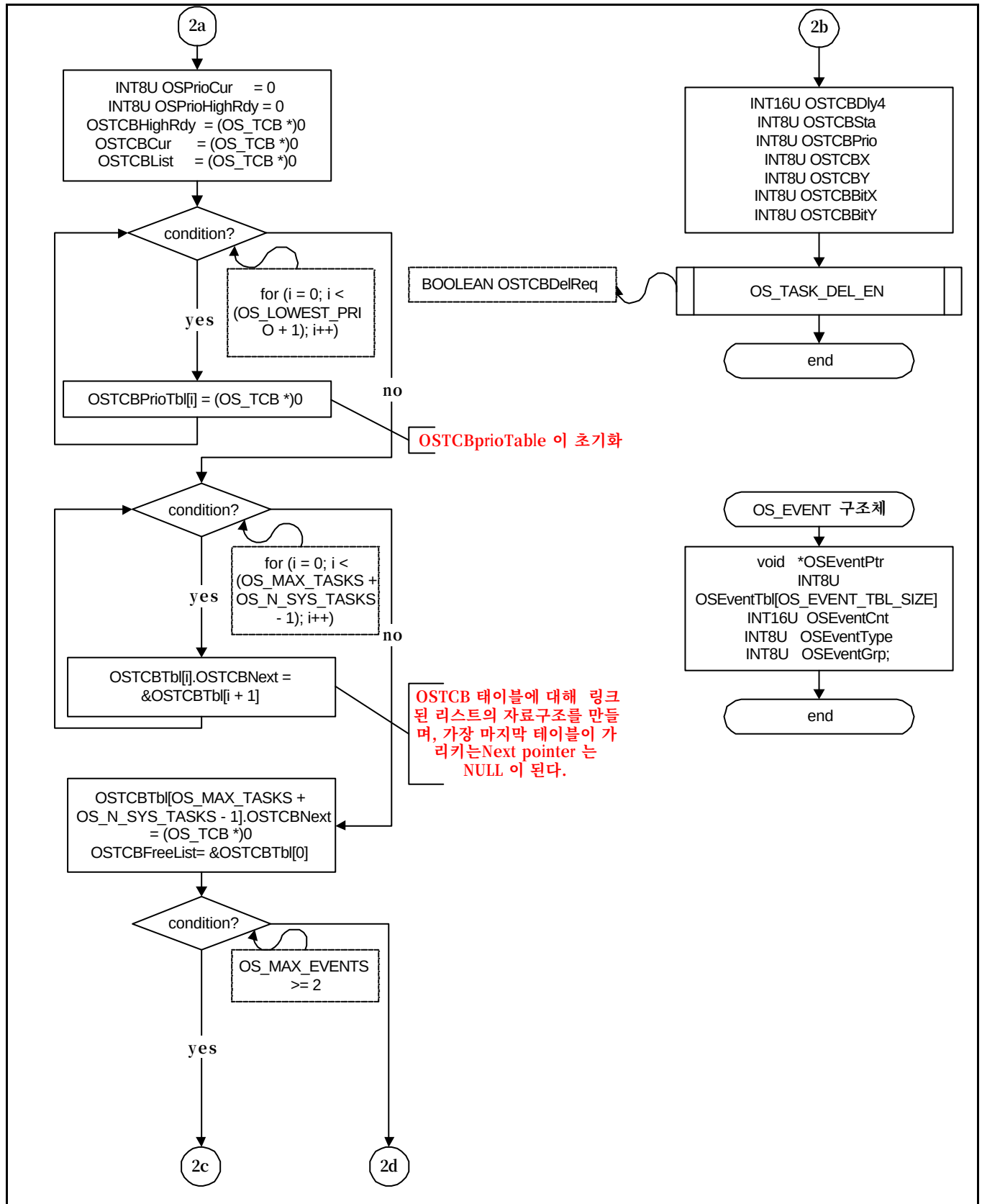
```

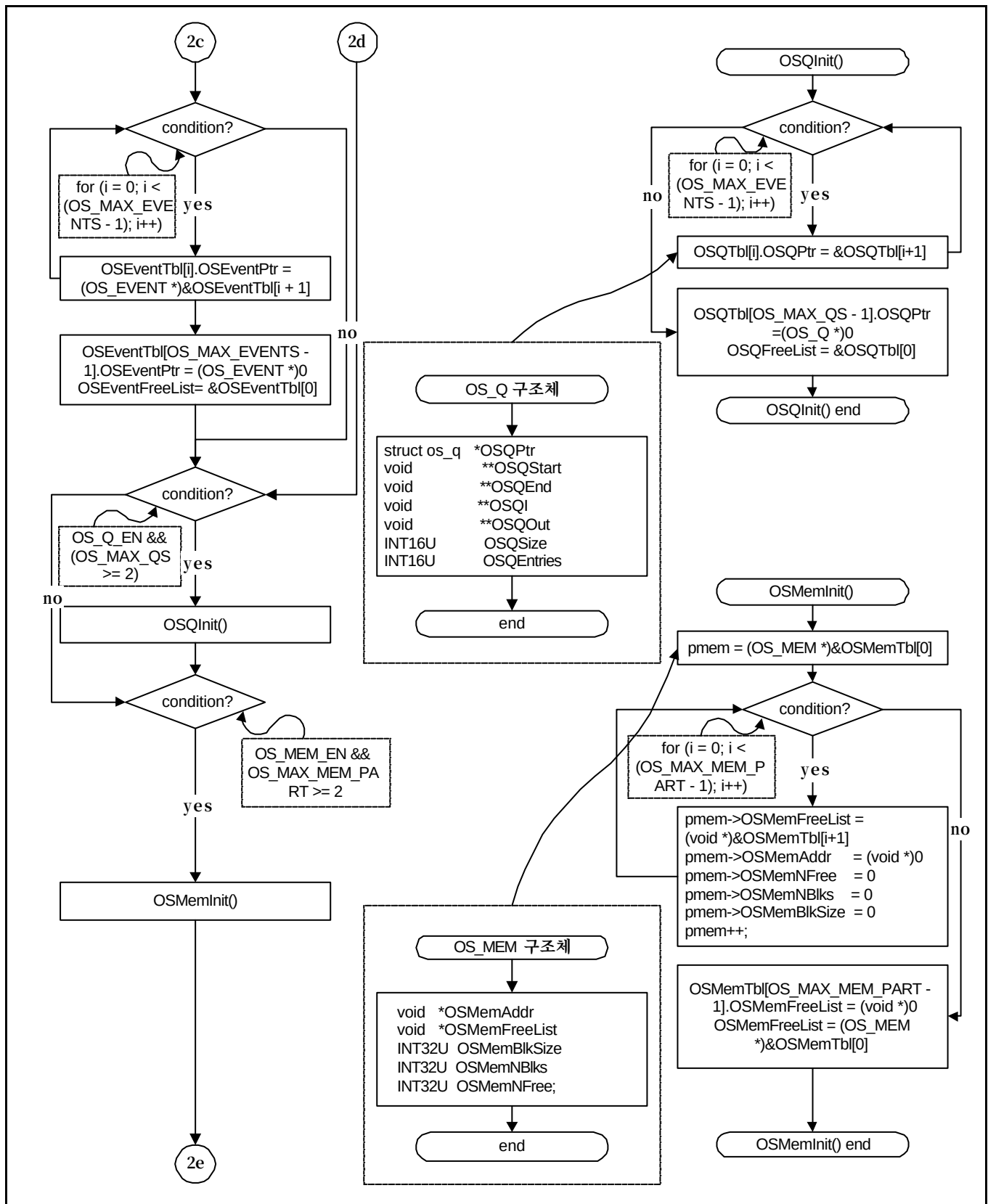
#define OS_MAX_EVENTS    2          /*최대 event control block(ECB)의 개수를 의미하며 2 이상의 값을 가져야한다*/
#define OS_MAX_MEM_PART  2          /*최대 Memory partition의 개수. 최소 2 이상*/
#define OS_MAX_QS        2          /*최대 queue control blocks의 개수. 최소 2 이상*/
#define OS_MAX_TASKS     11         /* 최대 tasks의 수 최소 2 이상*/
#define OS_LOWEST_PRIO   12         /* 가장 낮은 우선순위 지정. 작을수록 높은 우선순위이며 63보다 클 수 없다 */
#define OS_TASK_IDLE_STK_SIZE 512   /* Idle task stack size , 16비트 크기 */
#define OS_TASK_STAT_EN   1         /* 통계task(1초마다 실행되는 Cpu Usage 계산 테스트) 사용가능여부설정 */
#define OS_TASK_STAT_STK_SIZE 512   /* 통계 task stack size , 16비트 크기 */
#define OS_CPU_HOOKS_EN   1         /* uC/OS-II hooks are found in the processor port files */
#define OS_MBOX_EN        0         /* MAILBOX에 관련된 코드를 추가한다 */
#define OS_MEM_EN         0         /* 메모리 관리코드를 추가한다 (fixed sized memory blocks) */
#define OS_Q_EN           0         /* QUEUES에 관한 코드를 추가한다 */
#define OS_SEM_EN         1         /* SEMAPHORES에 관한 코드를 추가한다 */
  
```

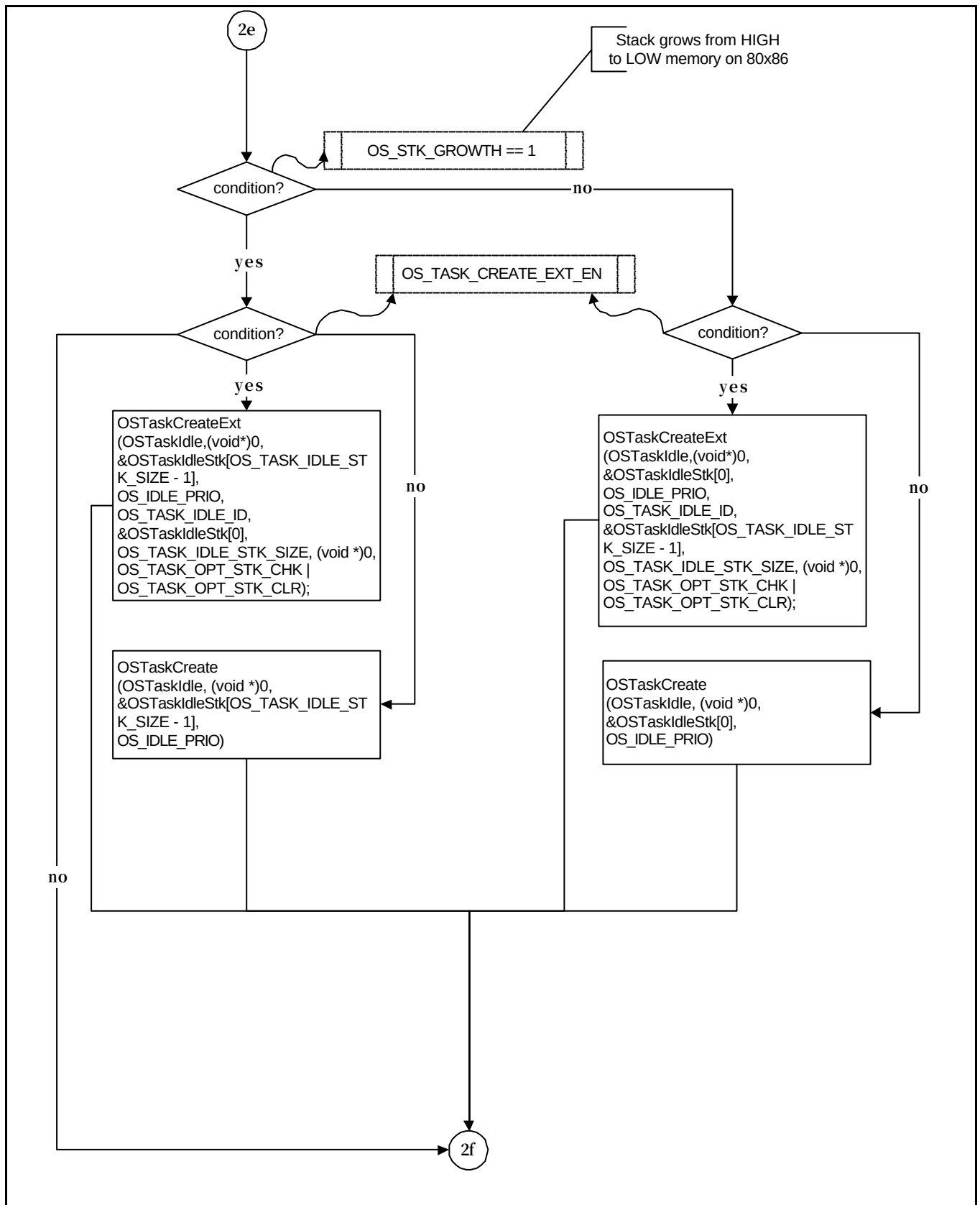
```
#define OS_TASK_CHANGE_PRIO_EN 0 /* OSTaskChangePrio()에 관한코드를 추가한다 */
#define OS_TASK_CREATE_EN 1 /* OSTaskCreate()에 관한코드를 추가한다 */
#define OS_TASK_CREATE_EXT_EN 0 /* OSTaskCreateExt()에 관한코드를 추가한다 */
#define OS_TASK_DEL_EN 0 /* OSTaskDel()에 관한코드를 추가한다 */
#define OS_TASK_SUSPEND_EN 0 /* OSTaskSuspend() and OSTaskResume()에 관한코드를 추가한다 */
#define OS_TICKS_PER_SEC 200 /* 1초당 tick의 개수를 설정한다 */
```

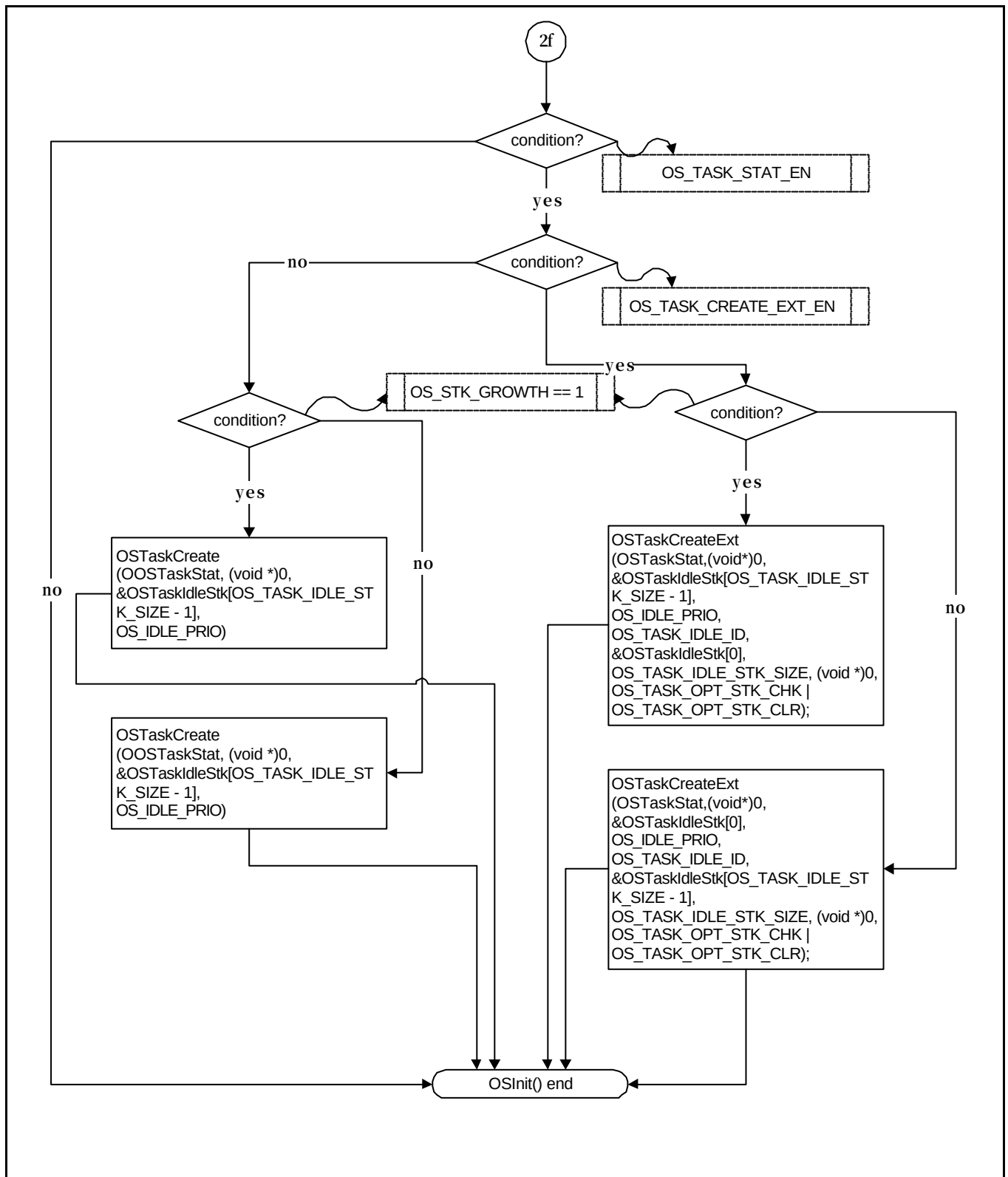
2 Program Flowchart (EX11.exe)











< Figure 1-2 OSInit(), PC_DisPClrScr () FlowChart >

위의 프로그램 흐름도는 처음 ex1l.c의 main() 함수에서부터 시작하여 void PC_DispcrScr (INT8U color) 함수와 OSInit() 함수의 실행과정을 그린 것이다.

1. void PC_DispcrScr (INT8U color)

이 함수는 VIDEO RAM을 직접 액세스 함으로서 PC 스크린을 clear 시키는 기능을 수행한다. 리턴 값은 없으며 컬러를 지정하는 8비트 인수 값을 하나 갖는다. 만약 현재 사용자가 사용하는 Video Card가 VGA 호환이라고 가정한다면 Video RAM은 B800 번지가 Base segment가 되며 B800:xxxx 주소에서 부터 스크린에 2Byte단위로 문자와 색깔을 표시하게 된다.

이 함수안에서 호출되고 있는 MK_FP(DISPCR_BASE, 0x0000)함수는 dos.h함수에서 아래와 같이 정의 되어 있다

```
#define MK_FP( seg,ofs )( (void _seg * )( seg ) +( void near * )( ofs ) )
```

위에서 보는 것처럼 MK_FP(함수로 정의된 _seg 라는 어셈블리 코드는 2개의 인수(Base segment, Offset address)를 이용하여, 실제 물리적 주소로 변환하는 계산을 한다고 생각된다.

2.void OSInit (void)

이 함수는 앞에서 말한 os_cfg.h 함수에서 선언된 여러 변수들을 이용하여 처음 초기화 할 변수와 함수들을 결정하는 일을 한다. 다음은 초기화하는 여러 변수 중 앞으로 중요하게 사용되는 몇 가지 변수들에 대해 설명한다.

(extern INT8U) OSRdyTbl[OS_RDY_TBL_SIZE]

Task가 처음 생성되면 Ready 상태로 들어가게 되는데, 각 task는 자기 자신의 priority를 ID로 하여 ReadyTable을 할당 받게 된다. 위의 Table배열의 크기는 미리 설정된 가장 낮은 priority에 의해 결정된다.

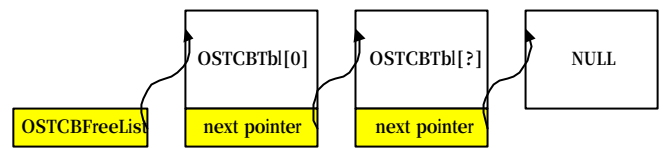
(struct) OS_TCB

처음 task가 생성되게 되면 각 task들은 모두 Task Control Block을 할당 받게 된다. 이 구조체에는 여러가지 변수들이 있는데 이 중 몇 가지를 예로 들면

‘스택의 시작과 끝의 Pointer’, ‘Next/Previous TCB pointer’, ‘Task Priority’, ‘TCB bit position’ 등이 있다. 이 변수 외에도 여러 가지 것들이 더 있으므로 1-4 page의 Flow chart를 참고하기 바란다.

(*OSTCB) OSTCBtbl[]

(OS_MAX_TASKS + OS_N_SYS_TASKS) 수 만큼의 길이를 갖는 (*OSTCB) 구조체 타입의 Table이다. Linked list 자료구조를 이용하여 next tcb table을 포인트 한다. 마지막 table의 next pointer는 ‘NULL’ 값을 갖는다. 변수 OSTCBFreeList 는 &OSTCBtbl[0] 주소를 포인트한다.

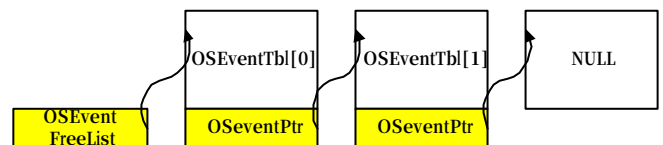


(OS_TCB *) OSTCBPrioTbl[]

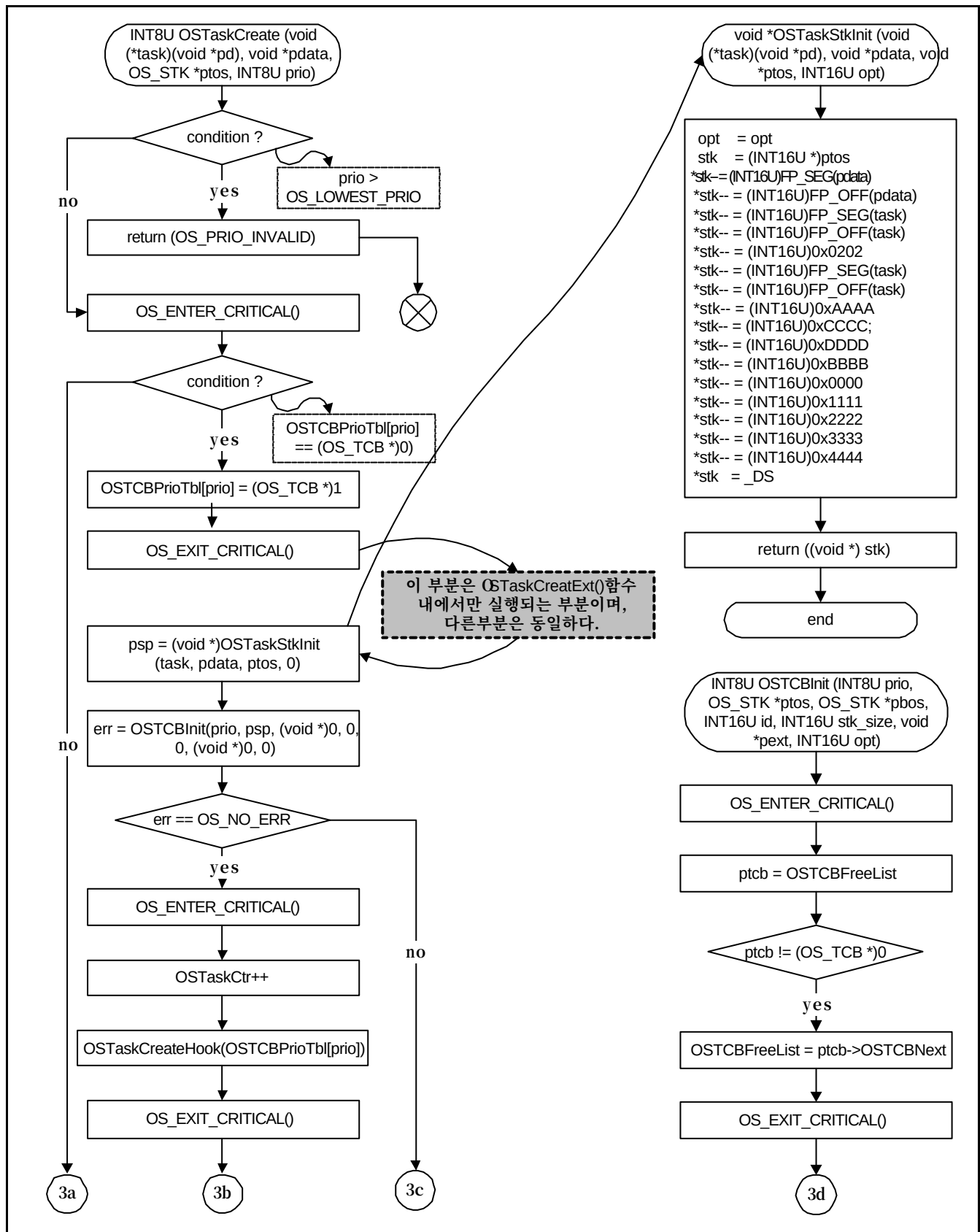
역시 위의 OSTCBtbl과 같은 (OS_TCB)구조체 타입 으로서, Task 마다 가지고 있는 TCB구조체를 포인트 하는 변수 이다. os_cfg.h에서 미리 설정된 최대 Task의 개수만큼 OSTCBPrioTbl[]를 만들고 이를 모두 ‘NULL’ 포인터로 초기화한다. 단지 priority를 이용하여 해당 Task의 TCB를 찾아내는 역할만을 수행할 뿐 linked list와 같은 자료구조 형식을 갖지는 않는다.

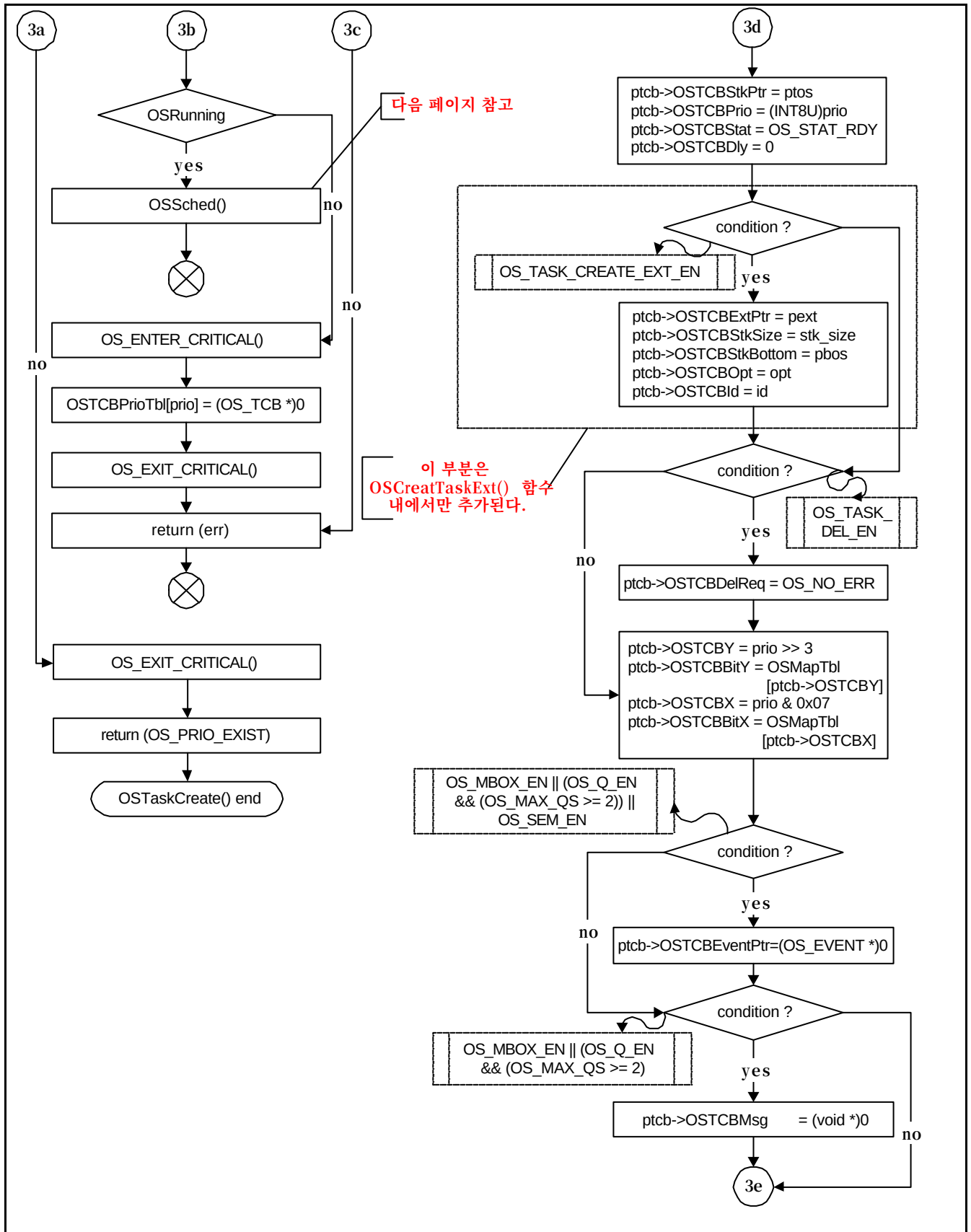
(OS_EVENT) OSEventTbl[]

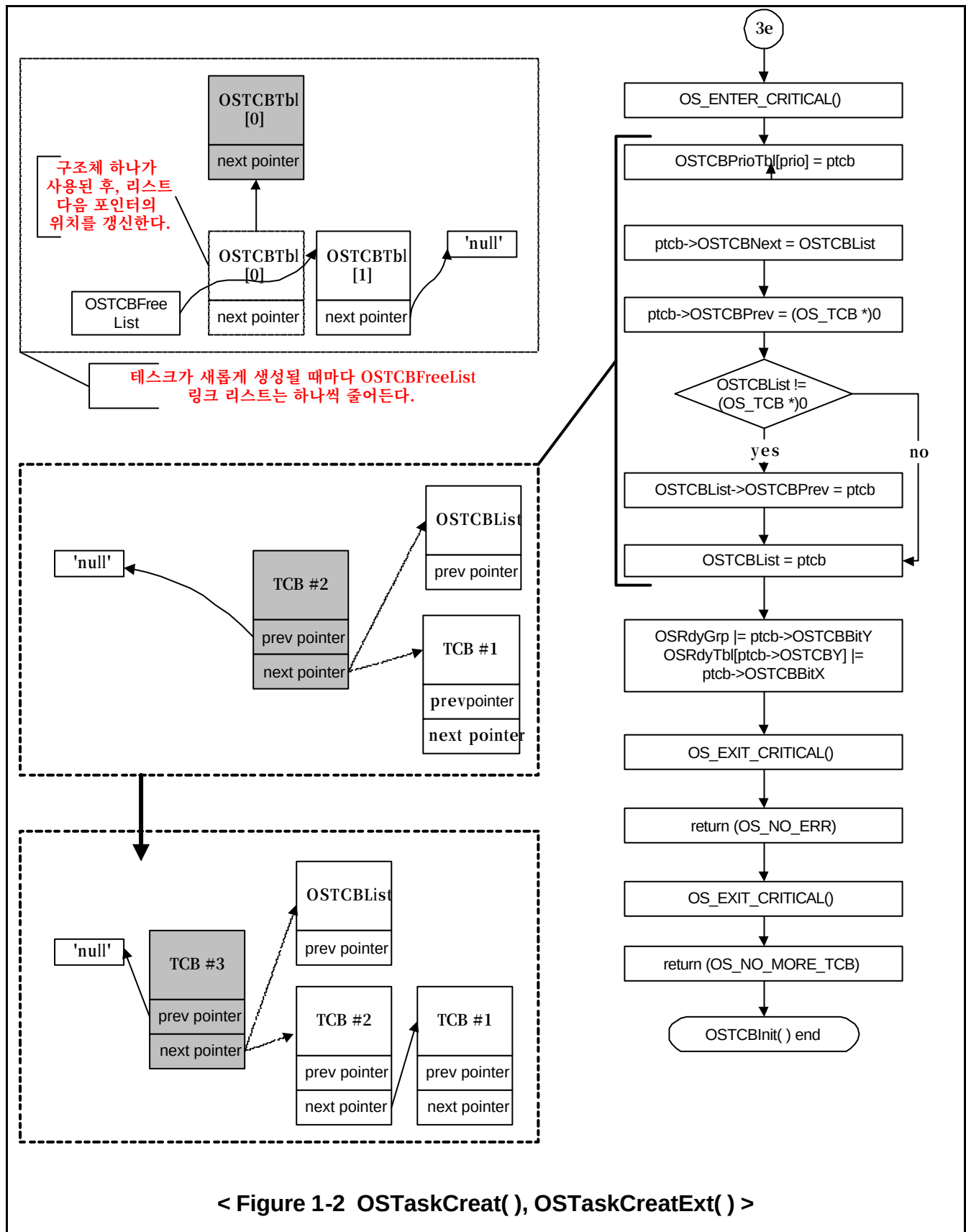
OS_EVENT 형식의 구조체타입(page 15참고) 으로서 미리 설정된 값만큼의 최대 OSEventTbl[] 배열을 생성하며, 그 형태는 아래와 같다.



지금까지 OSInit() 함수에서 초기화하는 여러 변수들에 대해서 설명하였다. 다음은 OSInit()함수 가장 마지막에서 호출되는 테스크 생성 함수 두 가지 OSTaskCreat(), OSTaskCreatExt() 그리고 이 두 함수가 각각 처음 생성 시키는 두개의 함수 OSIdleTask() 와 OSStatTask() 함수에 대한 플로우 차트이다.







3. OSTaskCreat(), OSTaskCreatExt()

task의 'priority'는 그 원래의 기능 이외에도 그 테스크 고유의 ID로서의 역할도 동시에 하게 된다. 처음 나오는 if 문은 현재 생성시키려 하는 task의 priority가 이미 다른 task에 의해 사용 중 인지를 체크한다.

```
(void *)OSTaskStkInit(task, pdata, ptos, 0)
```

task의 스택 프레임을 생성한다. Task는 현재 task code의 포인터, pdata는 task가 실행되어질 때 task에게 전해지는 사용자 정의 데이터의 포인터, ptos는 스택의 최상위 위치를 가리키는 포인터이다.

stk = ptos
pdata(SEG)
pdata(OFF)
task(SEG)
task(OFF)
0x0202
task(SEG)
task(OFF)
AX
CX
DX
BX
SP
BP
SI
DI
ES
_DS

ptos가 가리키는 스택의 가장 처음을 가리키는 포인터가 함수의 리턴값이 된다. 중간은 0x0202는 interrupt를 enable시키는 Status Word이고 마지막의 _DS에는 현재 Data Segment값이 저장된다. Task가 성공적으로 생성되면 현재 총 task를 표시하는 OSTaskCtr를 하나 증가시킨다. 그리고 지금 OSRunning 상태 라면 현재 만들어진 task가 실행 될 수 있도록 OSSched() 함수를 수행한다.

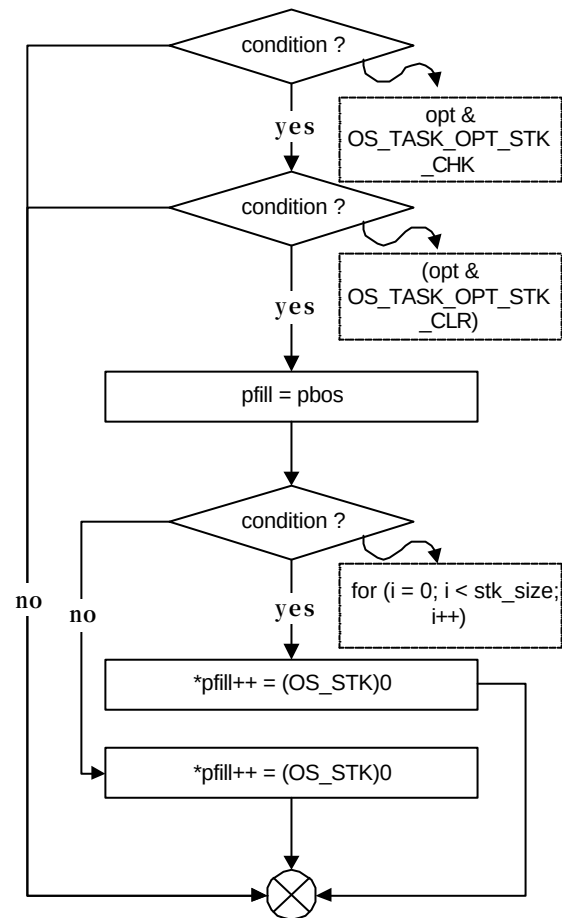
```
OSTCBInit(prio, psp, (void *)0, 0, 0, (void *)0, 0)
```

처음 OSInit() 에서(pase 1-5참고) 생성했던 OSTCB FreeList(= &OSTCBtbl[0]) 의 Linked list를 갱신하는 작업과, 새롭게 만들어진 TCB에 대해 OSTCB List라는 새로운 Linked list에 추가 시키는 일을 한다.

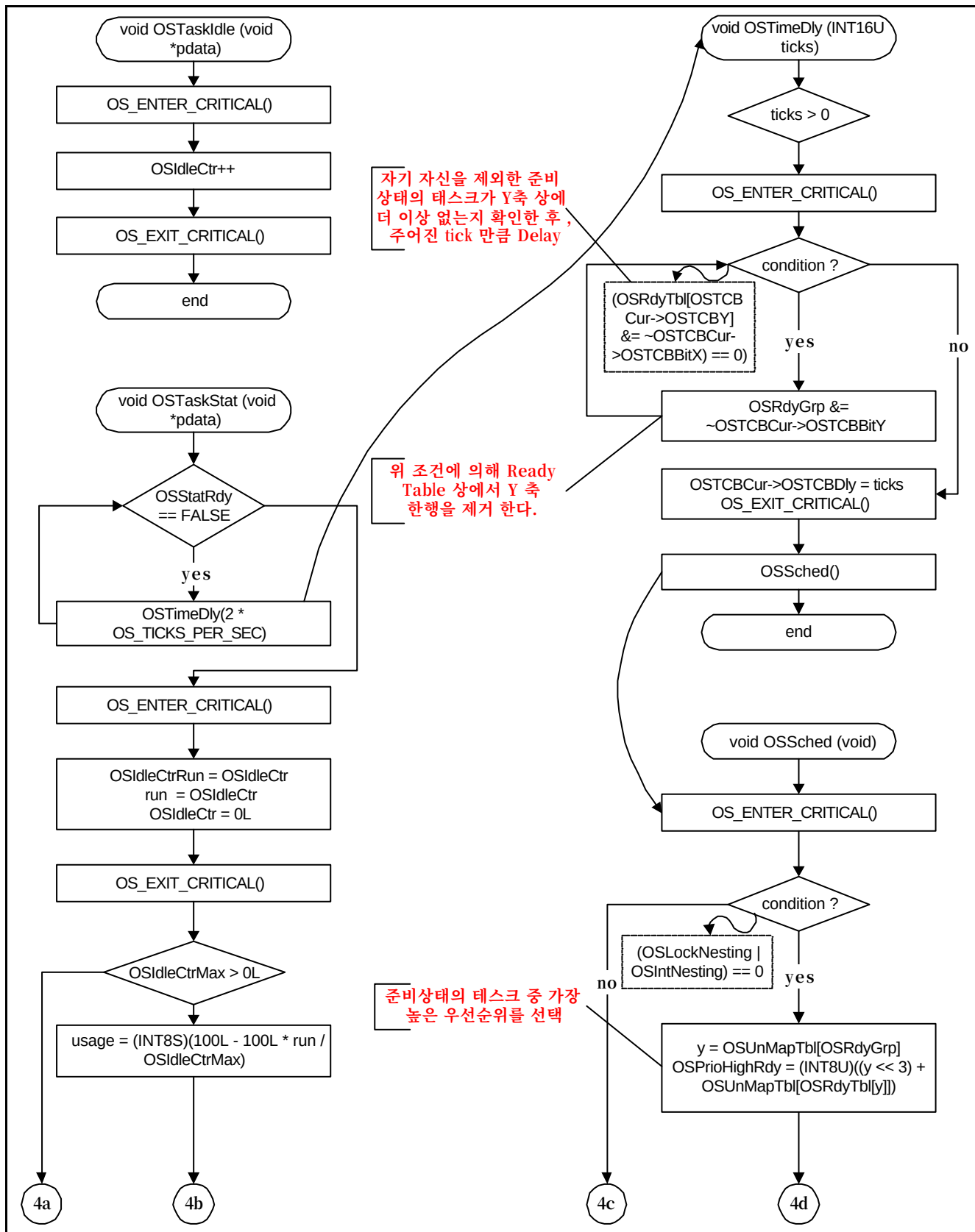
page 1-13 에 있는 왼쪽의 3 개의 블록 다이어그램은 그 과정을 나타낸 것이다.

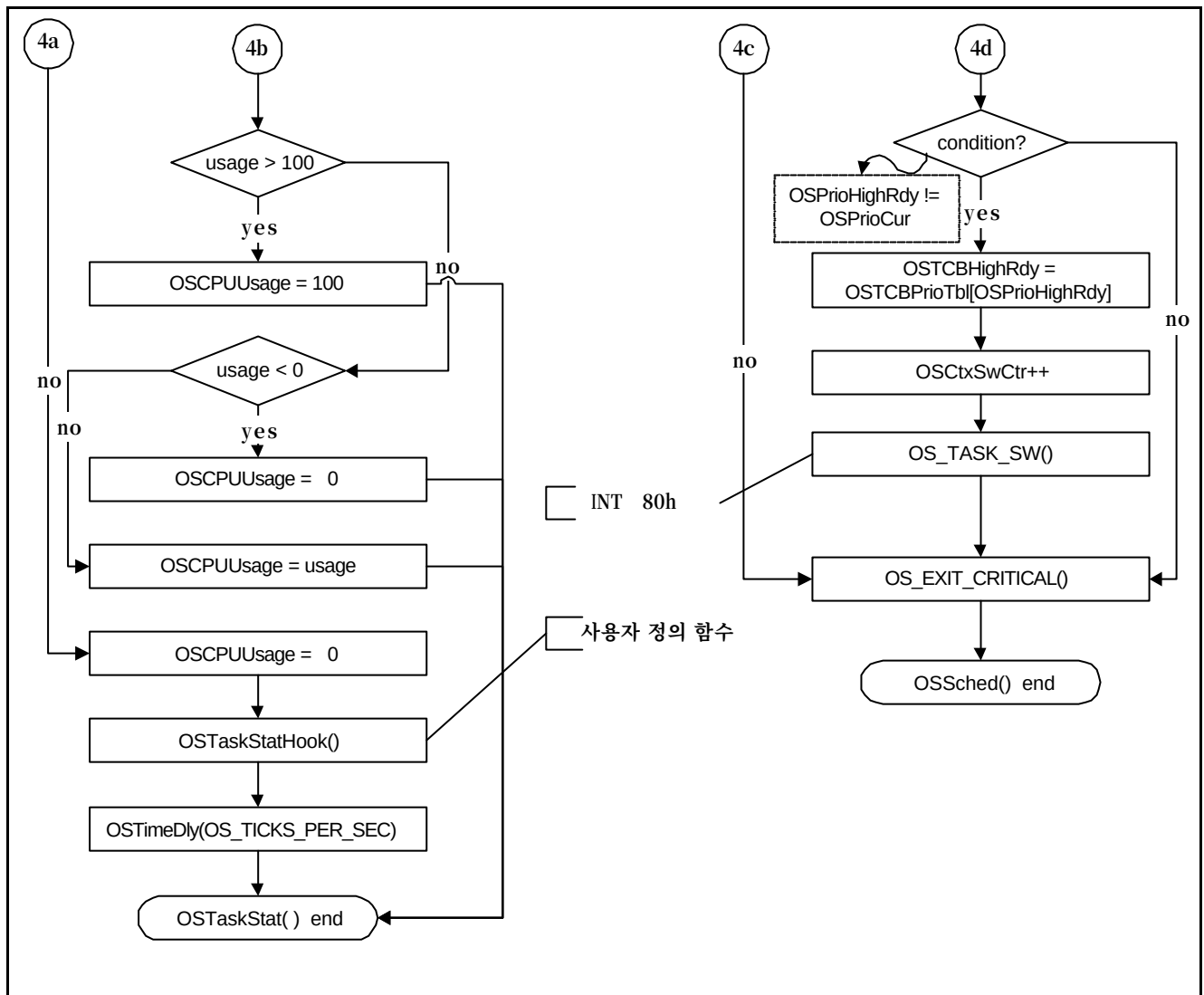
그리고 Semaphore, MessageBOX, Message Queue 등을 사용하는지 여부에 따라 추가적으로 TCB의 다른 변수들도 초기화하는 부분이 이어진다.

아래 그림은 OSTaskCreatExt() 함수에서만 실행되는 부분으로, 이외의 부분은 OSTaskCreat()함수와 완전히 동일하기 때문에 중복되지 않는 부분만 나타낸 것이다. 이는 11page의 중간 회색 블록에 추가된다.



다음 플로우 차트는 위에 설명한 task 생성 함수를 이용하여 가장 처음 생성하게 되는 TaskIdle()함수와 TaskStat()함수에 대한 것이다.





<Figure 1-3 OSTaskIdle(*pdata), void OSTaskStat(*pdata), OSTimeDly(ticks) >

4. OSTaskIdle(*pdata)

이 테스크는 가장 낮은 우선순위를 할당 받으며 단순히 IdleCtr++ 와 같이 카운터를 하나씩 증가 시키는 일만을 수행한다.

5. void OSTaskStat(*pdata)

이 함수는 1초에 한번씩 실행되면서 OStatRdy 변수가 TRUE상태가 될 때까지 일 때 까지 계속 2초간 대기하며, OSStart() 함수 내에서 OStatRdy 변수가 TRUE 상태로 바뀌게 되면 if 블록이 실행되면서, CPU 사용률을 계산하는 일을 수행한다

6. OSTimeDly(ticks)

현재의 task를 이 ticks수 만큼 딜레이 시키고 Ready Table에서 제거한다. (_OSTickISR() 에서 OSTimeTick()함수를 호출함으로서 실질적으로 수행되며 뒤에 다시 설명한다)

Void OSSched(void)

OSLockNesting, OSIntNesting 변수는 각각 현재 OS의 상태가 스케줄링이 진행되고 있는 중인지 또는 ISR 레벨에 있지않은지를 확인하는 변수이다. OSLO 차 Nesting 변수는 OSSchedLock (void) 함수가 실행 될때 1로 증가하며, OSIntNesting 변수는 OSInt Enter(void), OSIntExit(void) 함수에서 증가 또는 감소하게 된다.

```
Y=OSUnMapTbl[OSRdyGrp]
OSPrioHighRdy=(INT8U)((y << 3) + OSUnMapTbl[OSRdyTbl[y]])
```

위의 두 줄은 현재 Task Ready Table에서 가장 높은 우선순위를 찾아내는 알고리즘이다. 가장 우선순위가 높은 task가 선택되면 현재 실행중인 Task의 우선 순위와 비교 후 다르다면 이 두 Task를 Context Switching하는 ISR루틴을(int 80h) 실행한다

아래 그림은 우선순위를 찾는 알고리즘을 그림으로 그림 것이다. 아래의 그림을 현재 총 5개의 우선 순위를 가지는 테스트들이(노란색) 준비 상태에 있다는 것을 알 수 있다. 처음 테스트가 생성 되면 INT8U OSTCBInit() 함수가 자신이 할당 받은 TCB구조체 내의 여러 변수들 초기화하게 되는데, 이 함수 내에서 다음의 코드가 우선순위에 관련된 4가지 변수 값을 초기화 시키게 된다.

```
ptcb->OSTCBY = prio >> 3
ptcb->OSTCBX = prio & 0x07
ptcb->OSTCBBitY = OSMaPtbl[ptcb->OSTCBY]
ptcb->OSTCBBitX = OSMaPtbl[ptcb->OSTCBX]
```

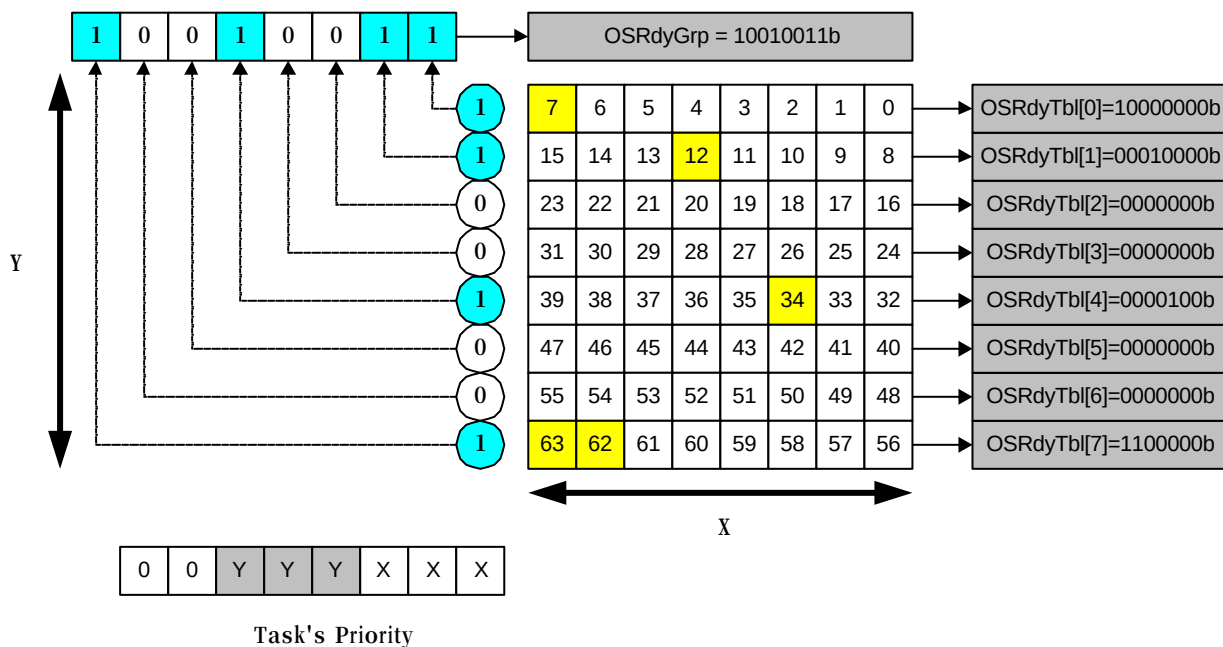
예를 들어 방금 우선순위 12 번의 task 가 생성되었다고 가정하면 각 TCB 구조체의 변수들은 다음과 같이 초기화 될 것이다.(priority 12= 0000 1100b)

```
ptcb->OSTCBY = 0000 0001b
ptcb->OSTCBBitY = 0000 0001b
ptcb->OSTCBX = 0000 0100b
ptcb->OSTCBBitX= 1000 0000b
```

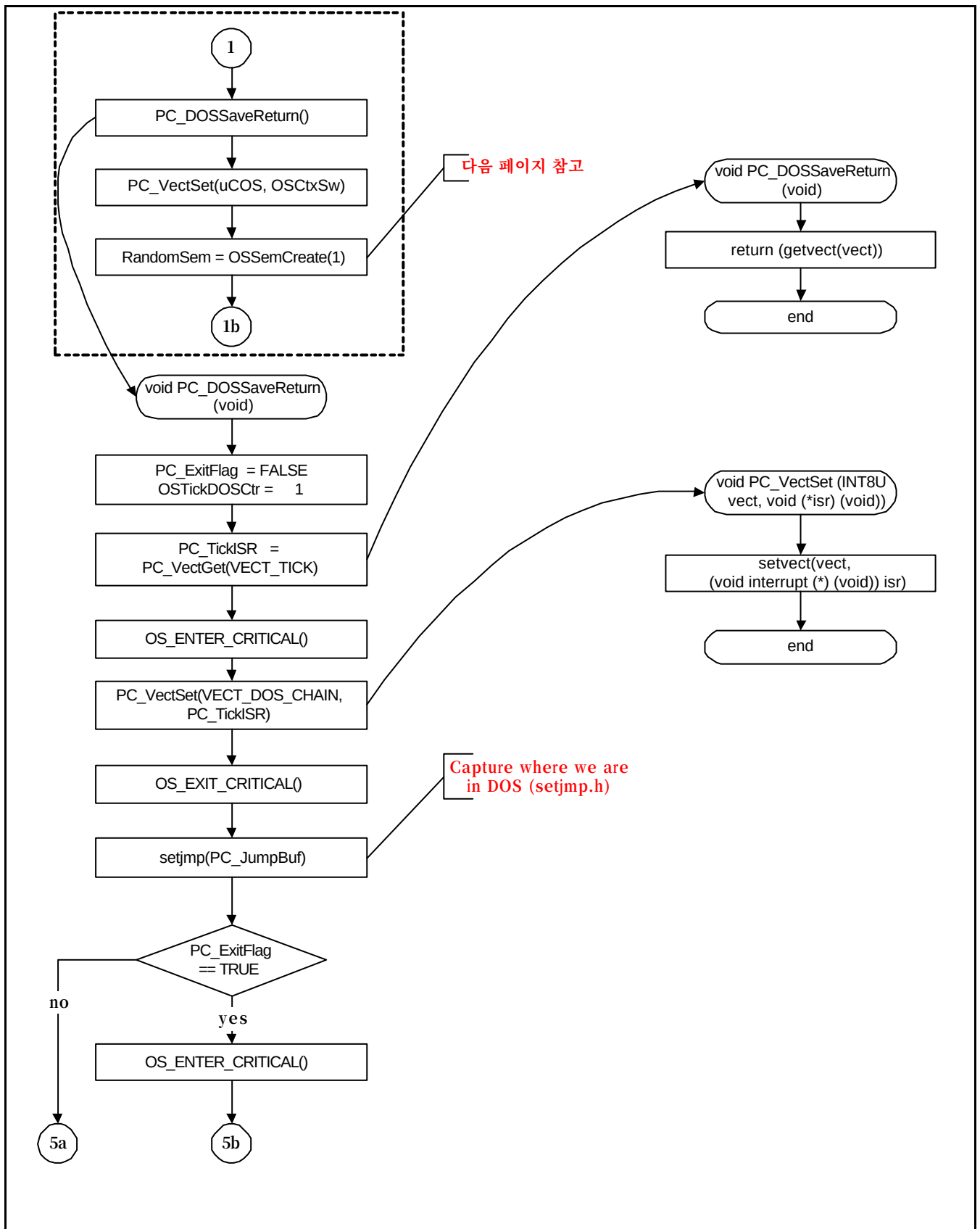
위에서 진하게 표시된 부분은 각각 OSRdyGrp 변수와 OSTCB[]변수에 OR 연산을 거쳐 계속 누적된다.

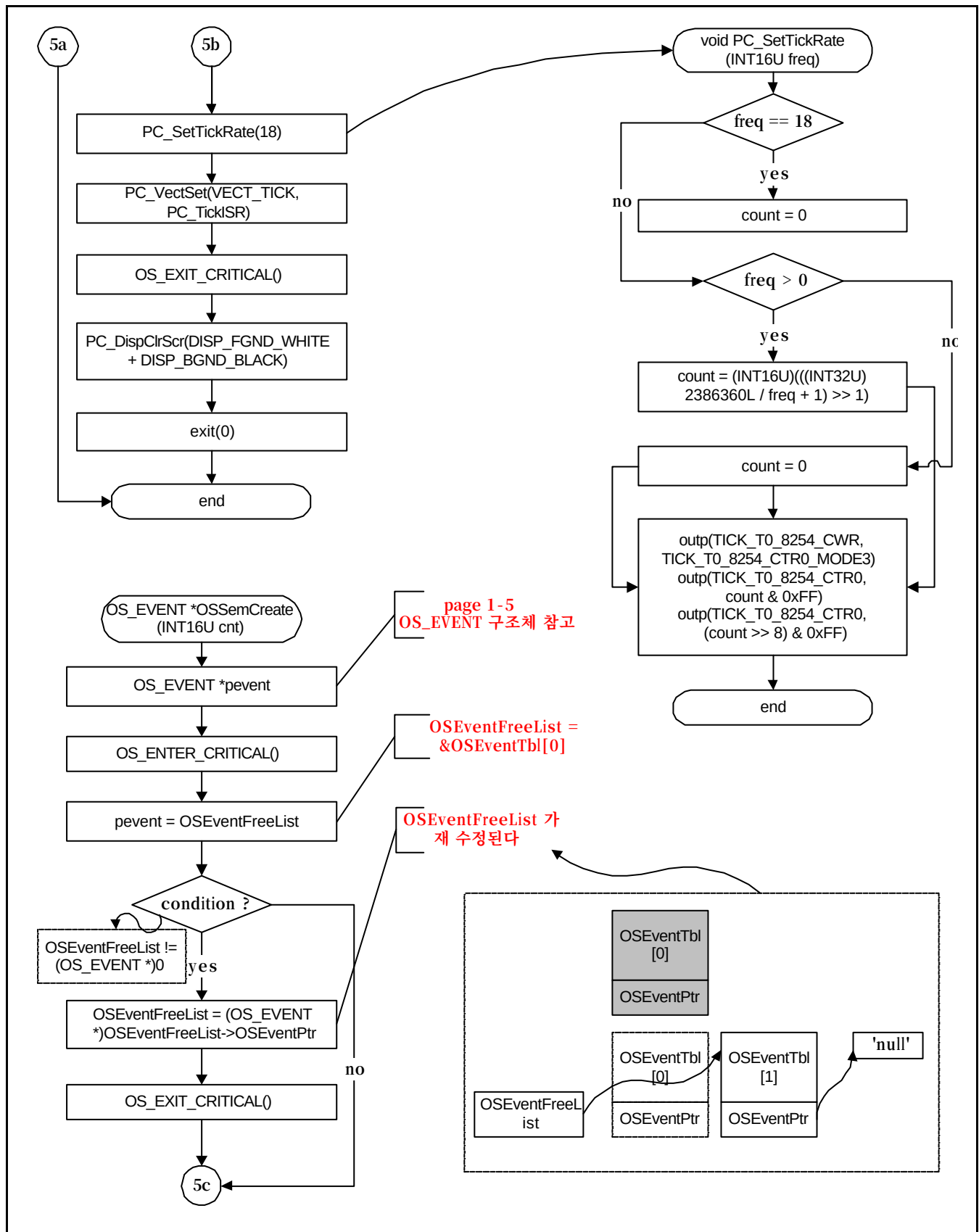
```
OSRdyGrp |= ptcb->OSTCBBitY
OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX
```

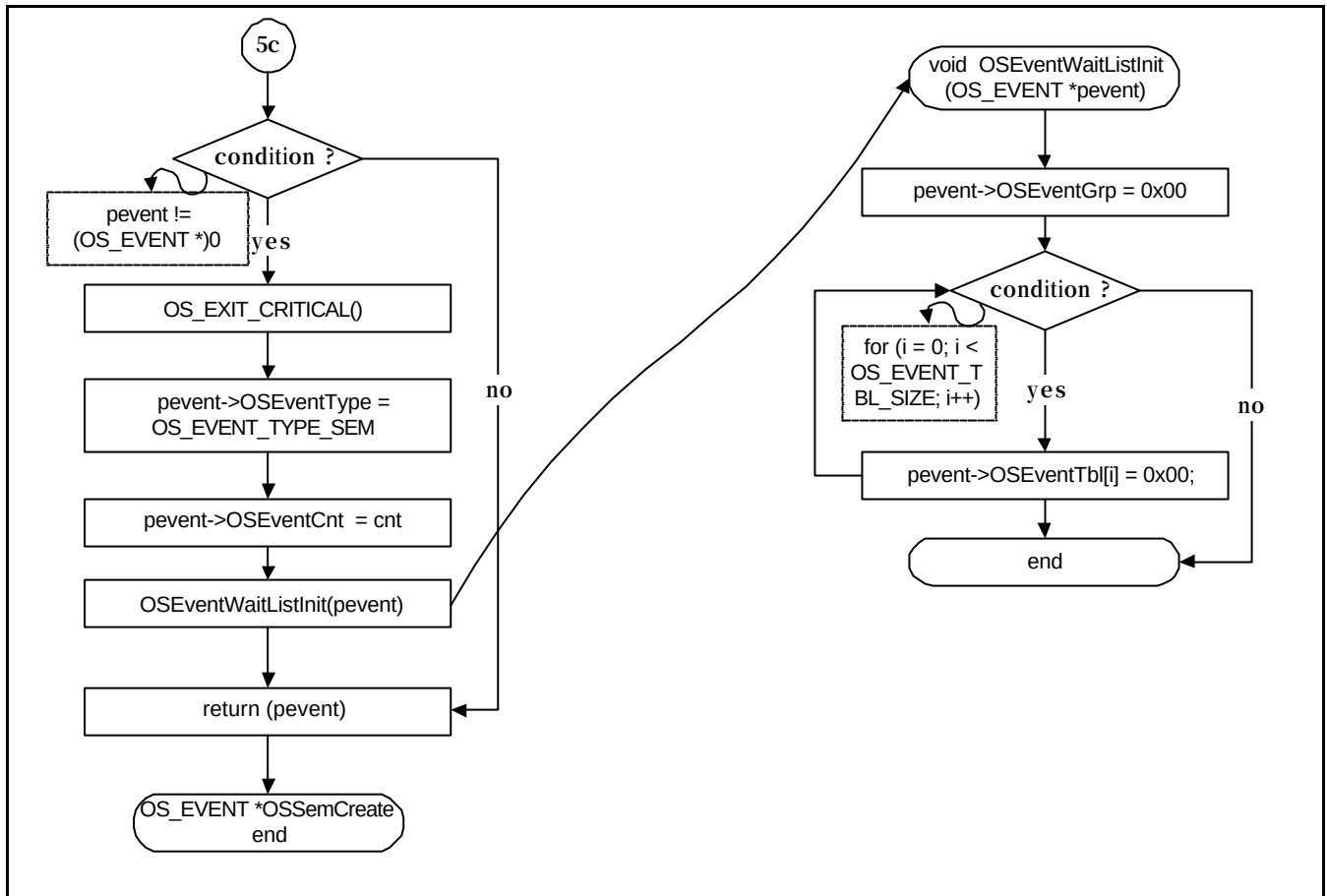
Task가 생성될 때마다 위의 과정을 거쳐 OSRdyGrp 변수와 OSRdyTbl[] 변수 값은 계속 누적되며, 이를 UnMapTbl[OSRdyGrp] 을 거쳐 현재 현재 Ready Table 에서 가장 높은 우선순위 테스트의 Y 값을, 이를 다시 OSUnMapTbl[OSRdyTbl[y]]을 통해 X 값을 구할 수 있게 된다. OSUnMapTbl[] 은 OSRdyGrp의 255가지 경우의 수에 대해 가장 상위의 가로줄 위치를 찾을 수 있도록 미리 만들어 놓은 리스트이다. 여기서 구한 X, Y값을 이용하여 재 배치하면 가장 높은 우선순위의 테스트를 쉽게 찾을 수 있다.



< Figure 1-4 우선순위를 찾는 알고리즘 >







< Figure 1-5 PC_DOSSaveReturn(), OS_SemCreate() >

7. PC_DOSSaveReturn()

이 함수는 uC의 실행이 종료되었을 때 우리가 처음 이 프로그램을 실행시키기 직전의 위치로 다시 돌아가기 위한 주소를 저장하는 함수이다.

보통 일반적인 프로그래밍을 할 때에는 이러한 함수를 사용하지 않아도 되지만 uC의 경우에는 마지막으로 호출되는 OSStart() 함수가 한번 실행되게 된 후 다시는 이 함수가 처음 실행되었던 위치로 돌아오지 못하기 때문에 임의로 현재의 스택환경을 버퍼에 미리 저장하고 종료 시 다시 이 버퍼에 저장된 스택 값들을 복구 함으로서 정상적으로 DOS로 복귀할 수 있도록 하는 역할을 한다. 처음 main() 함수가 호출되면, 이 함수는 PC_ExitFlag를 FALSE로 설정하며, 이어지는 'if' 문은 실행하지 않을 것이다. 대신 스택 환경들이 'PC_JumpBuf' 라는 buffer에 저장되며, ESC키가 입력될 때, PC_DOSReturn() 함수에서 PC_ExitFlag가 TRUE로 재 설정되고 'if' branch문이 실행되면서 함수를 종료한다.

PC_VectGet(VECT_TICK)

이 함수는 아래와 같이 dos.h에 정의되어 있는 getvect() 함수를 다시 호출한다.

```
void interrupt( far * _CType getvect( int __interruptno ) )();
```

참고 'interrupt' 는 이 함수가 ISR(Interrupt Service Routine)임을 타나 내는 keyword이며, 이는 이 함수가 일반 calling convention과는 달리 함수 실행 종료 후 복귀 시 특별한 리턴 명령어와(IRET), 모든 레지스터 들이 PUSH, POP 되는 등의 인터럽트 루틴만이 가지는 특별한 코드가 컴파일 시 더 추가 되어 함을 의미한다. 그리고 CType keyword는 헤더파일을 뒤져보면 'cdecl' keyword로 다시 정의 되는데(_defs.h) 일반적인 stdcall keyword와 비슷한 call convention 에서 누가 호출시 사용한 스택을 클리어 할 책임을 가지는가에 따라 구분된다. 이러한 keyword는 컴파일러마다 자기만의 방식으로 새롭게 확장한 경우가 많기 때문에 (under bar를 두개 붙인다는 식으로) 각 컴파일러의 매뉴얼을 읽고 참고해 보아야 할 것이다. 마지막으로 이 함수는 void형 pointer인데, 포인터 타입이 void라는 의미는 const, volatile로 선언되지 않은 어떤 변수형이라도 포인터로 받을 수 있음을 의미한다. 이 함수에서는 인터럽트 핸들러 함수가 실제 위치한 주소를 vector table에서 참조하여 그 포인터 넘겨준다고 생각된다. (MSDN reference manual 참고.)

PC_VectSet(VECT_DOS_CHAIN, PC_TickISR)

이 함수는 아래와 같이 dos.h에 정의 되어있는 setvect() 함수를 다시 호출한다. 여기서는 dos의 tick ISR vector가 포인트 하는 값을 VECT_DOS_CHAIN (=81h)로 재 지정 함으로서 원래의 Dos Tick ISR의 실행에 문제가 없도록 한다.

```
void _CType setvect ( int __interruptno, void interrupt ( far * __isr )())
```

참고) 여기 나오는 keyword들 역시 앞에서 설명한 것과 동일한 것들이며 이 함수는 interrupt number와 ISR을 가리키는 포인터를 파라미터로 받는다. 리턴값은 없다.

setjmp(PC_JumpBuf)

setjmp는 스택환경을(register제외) 그대로 저장하며, 나중에 longjmp 명령어를 사용하여 저장된 스택이 그대로 복구된다. 이 두 명령어가 함께 사용될 때는 'non-logical goto'가 실행되는 효과를 갖으며 일반적인 calling 또는 return convention을 사용하지 않는 error-handling 또는 recovery code로 프로그램의 제어를 넘겨줄 때 주로 이용 된다고 한다. (MSDN reference manual 참고...)

PC_SetTickRate(freq)

이 함수에 사용된 2386360 이라는 숫자는 실제로 8254가 동작하는 주파수인 1.193180Mhz값에 2를 곱한 값이다. 이는,

$$\text{-counter} = (1193180.0 \text{ Hz/freq}) + 0.5$$

라는 계산과 같은 의미이다. 즉, 동작시키고자 하는 주파수 값만 써 넣으면 된다. 예를 들어 freq=1 이라는 값을 파라미터로 받는다면 카운터가 1Hz로 동작하며, freq=18로서 호출한 함수는 18.2Hz (55ms)의 도스가 기본적으로 사용하는 tick time으로 동작한다.

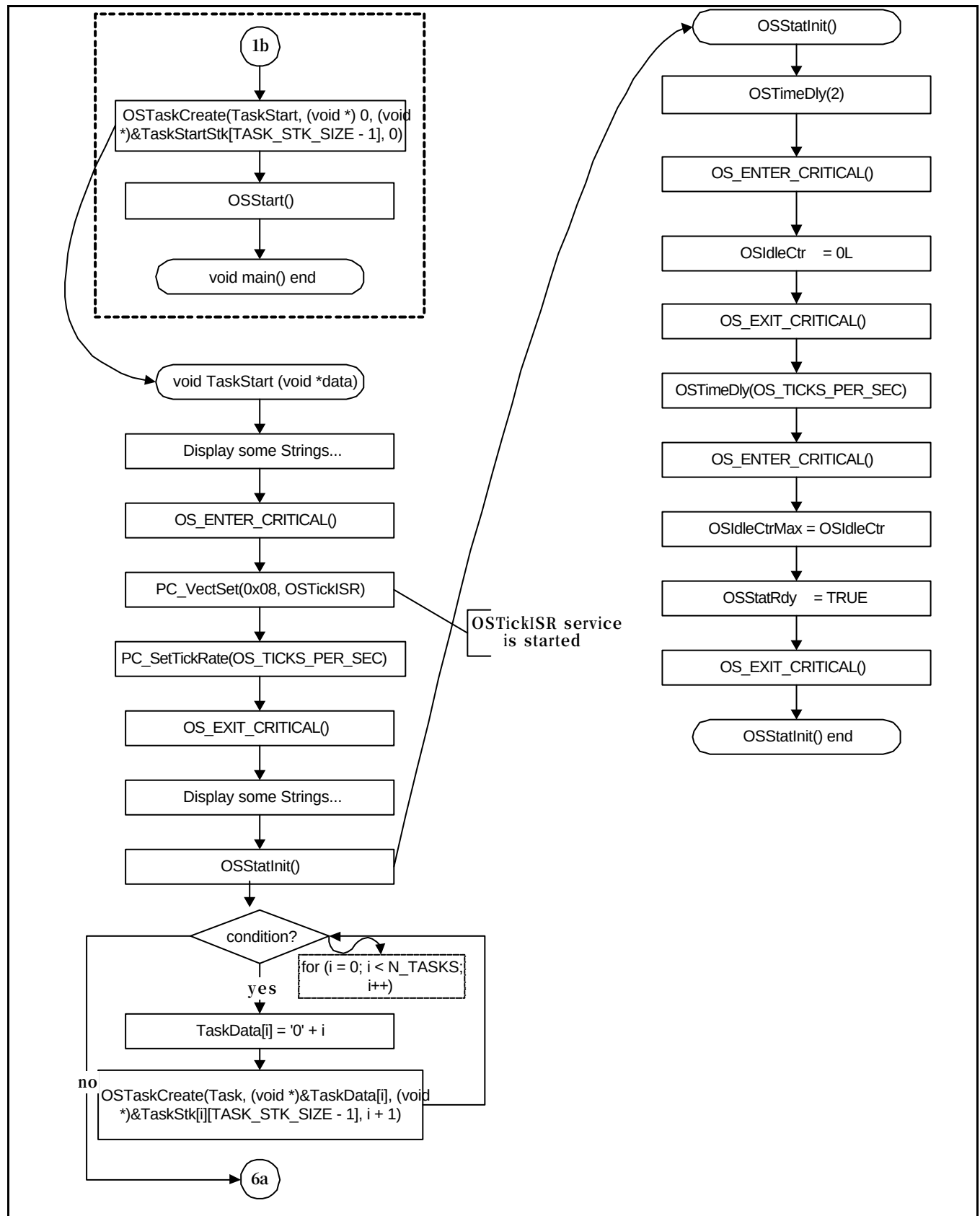
참고) 현재 표준 PC/AT와 호환성을 가지는 시스템의 경우 4kbyte크기를 가지는 총 15개의 슬롯으로 구성된 64k의 I/O address 공간을 갖는다. 하지만 예전의 1kbyte I/O address 자원을 사용하던 피시들과의 호환성을 고려하여, 각 슬롯의 최하위 256byte인 0x000~0xFF까지의 address는 플랫폼(마더보드) I/O 자원으로 할당하고, 나머지 0xFF ~ 0x3FF까지의 768byte는 외부 I/O address 자원으로 사용되며, 이는 모든 PC에서 호환성을 위해 동일한 구조를 갖는다. 이의 확장을 위해서 단순히 22-23h의 바이트 값에 특별한 비트를 set함으로서 확장된 슬롯의 I/O address가 사용가능해진다. 여기서 사용된 82c54 counter는 바로 하위 256byte의 한 부분인 0x40 ~ 0x43까지의 I/O register address를

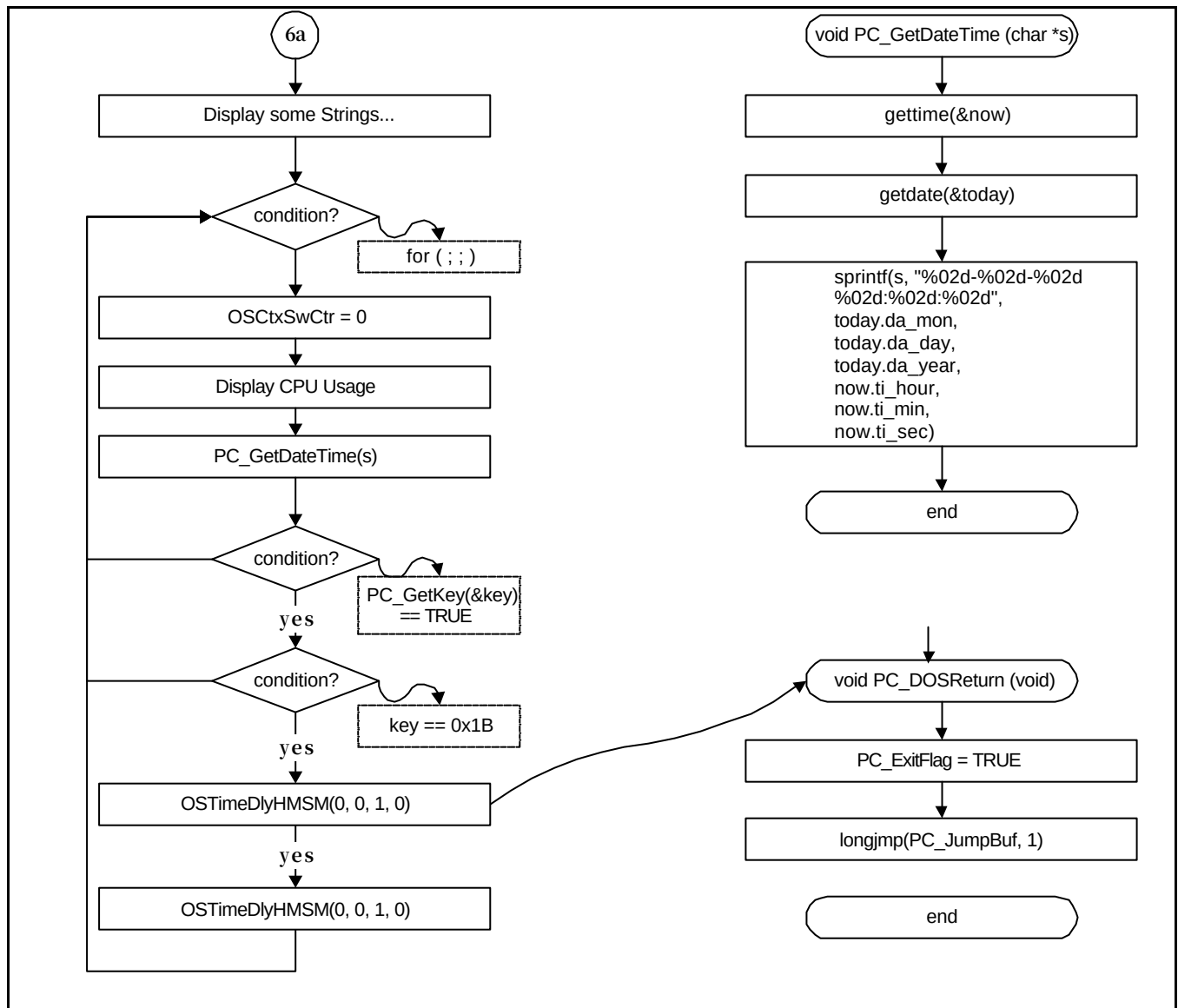
를 액세스함으로서 counter를 제어한다. 82c54의 Mode3 동작 방식이란 현재 입력된 counter에서 각 클럭마다 2씩 줄여나가는 방식으로 카운트 하는 것을 말한다. 출력 값은 처음 High 상태이며, 카운트 값이 줄어들어 0이 되면 출력은 High에서 Low로 바뀐다. 이때 다시 처음부터 counter를 시작하여 또다시 0이 되면, 다시 출력을 High로 바꾸는 과정을 계속 반복하게 된다.

8. OSSemCreate()

현재 OSEventFreeList를 검사하여 Free Event Control Block이 남아 있나 체크 한다. 이 리스트의 가장 마지막의 포인터는 NULL이기 때문에 이 조건이 맞지 않다면 OSFreeEventList에서 하나의 element가 삭제되면서 새로이 linked list는 갱신된다.

새롭게 생성된 ECB는 OSEventWaitListInit(pevent) 함수를 이용하여 새로운 Event Group을 생성한다. (이 알고리즘은 위에서 설명한 TCB의 Ready Table 지정방식과 거의 유사하므로 설명은 생략하도록 한다.

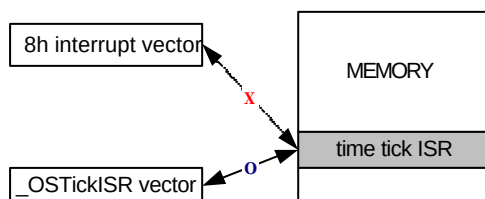




< Figure 1-6 OSTaskCreat(OSTaskStart(), .) >

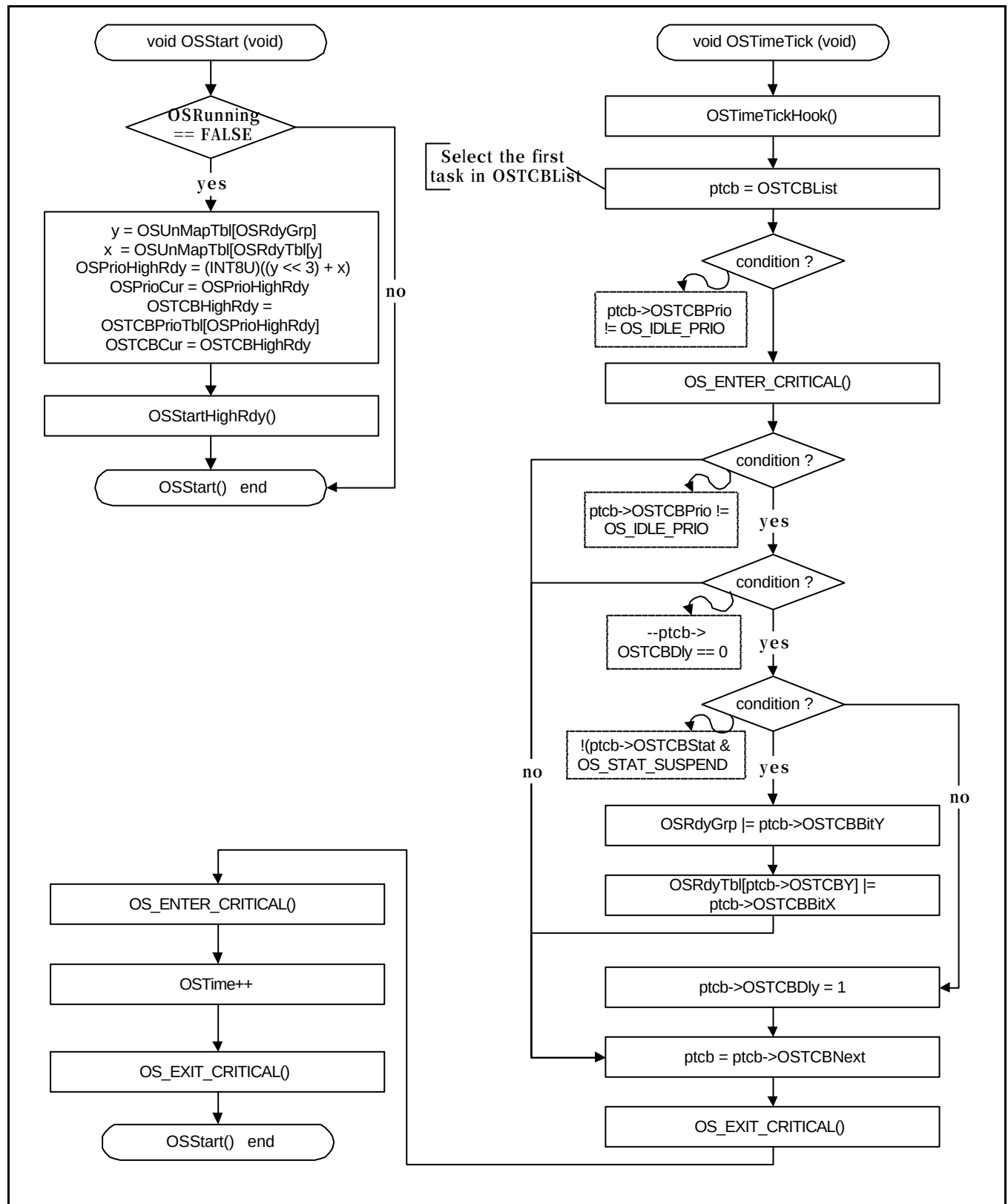
9. OSTaskStart()

여기에서는 TaskStart() 라는 세 번째 task가 생성된다. 이 함수는 0x8h번 DOS tick ISR를 OSTickISR() 함수와 연결(?) 시킨다. 즉, PC_SetTickRate() 함수에 의해 tick time 이 1/200 초로 설정 되면, 1초당 200번씩 OSTickISR() 함수가 Interrupt를 가로채어 실행된다.



OSStatInit()

Static(통계) 함수를 초기화 시킨다. 전체적인 과정을 설명하면, 이 초기화 과정에서 1초간 OSIdle() 함수를 실행시켜 OSIdleCtr가 1초동안 아무런 방해받지 않고 셀 수 있는 최대한의 카운터 OSIdleMax의 값을 구한 후, 이 후 이 값을 이용해 실제로 OSStart() 함수에서 멀티 테스킹에 들어갔을 때 카운트 되는 OSIdleCtr값의 비율을 비교하여 현재 CPU Usage를 계산하는 알고리즘으로 동작한다. 다만 지금 여기서는 OSTaskStart() 라는 task가 생성되는 과정이기 때문에, 실제 이 함수가 하는 일보다는 함수의 코드가 어떻게 구성되는지에 더 중점을 두어 간단히 설명하였다.



< Figure 1-7 OSStart(), OSTimeTick() >

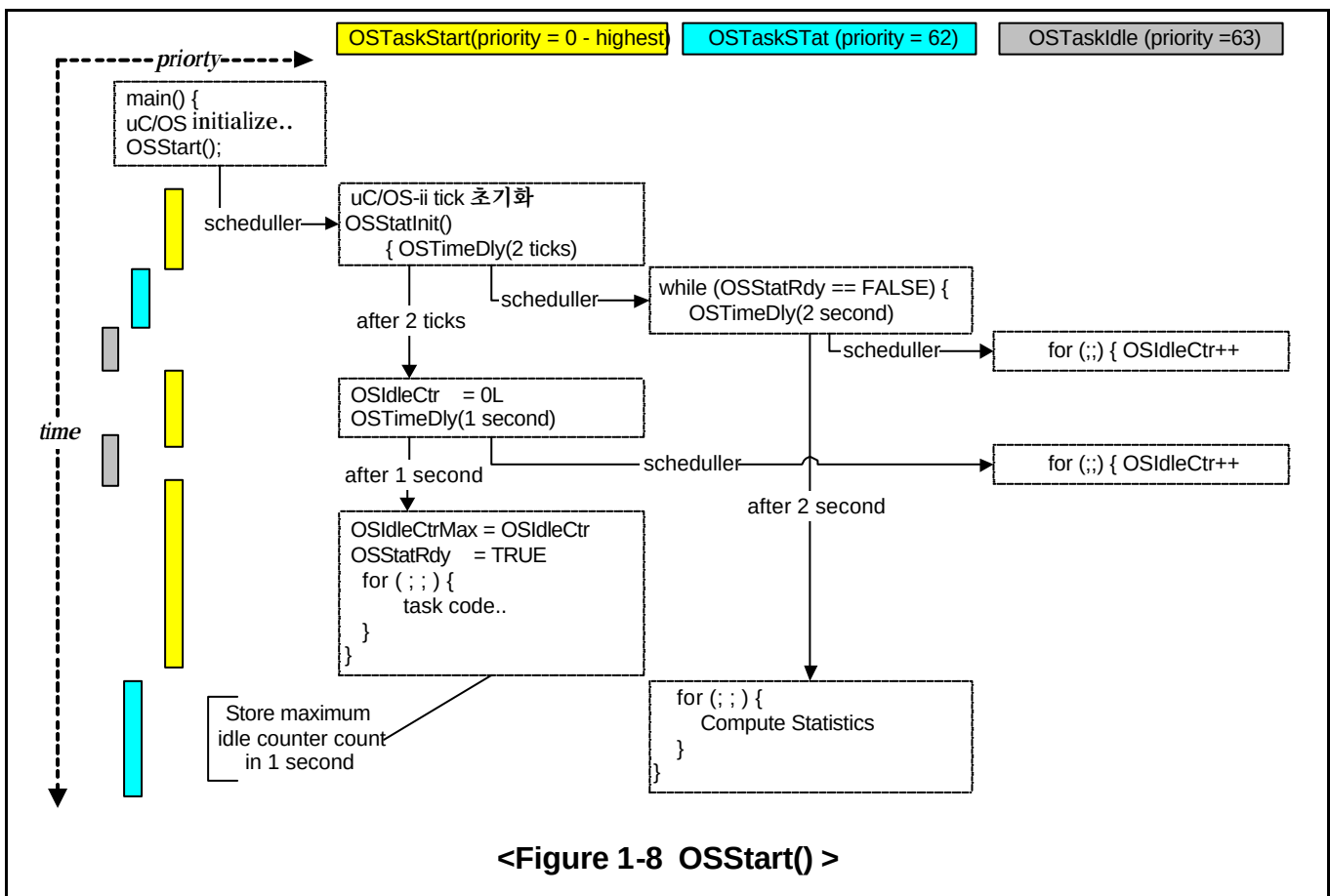
10. void OSStart(void)

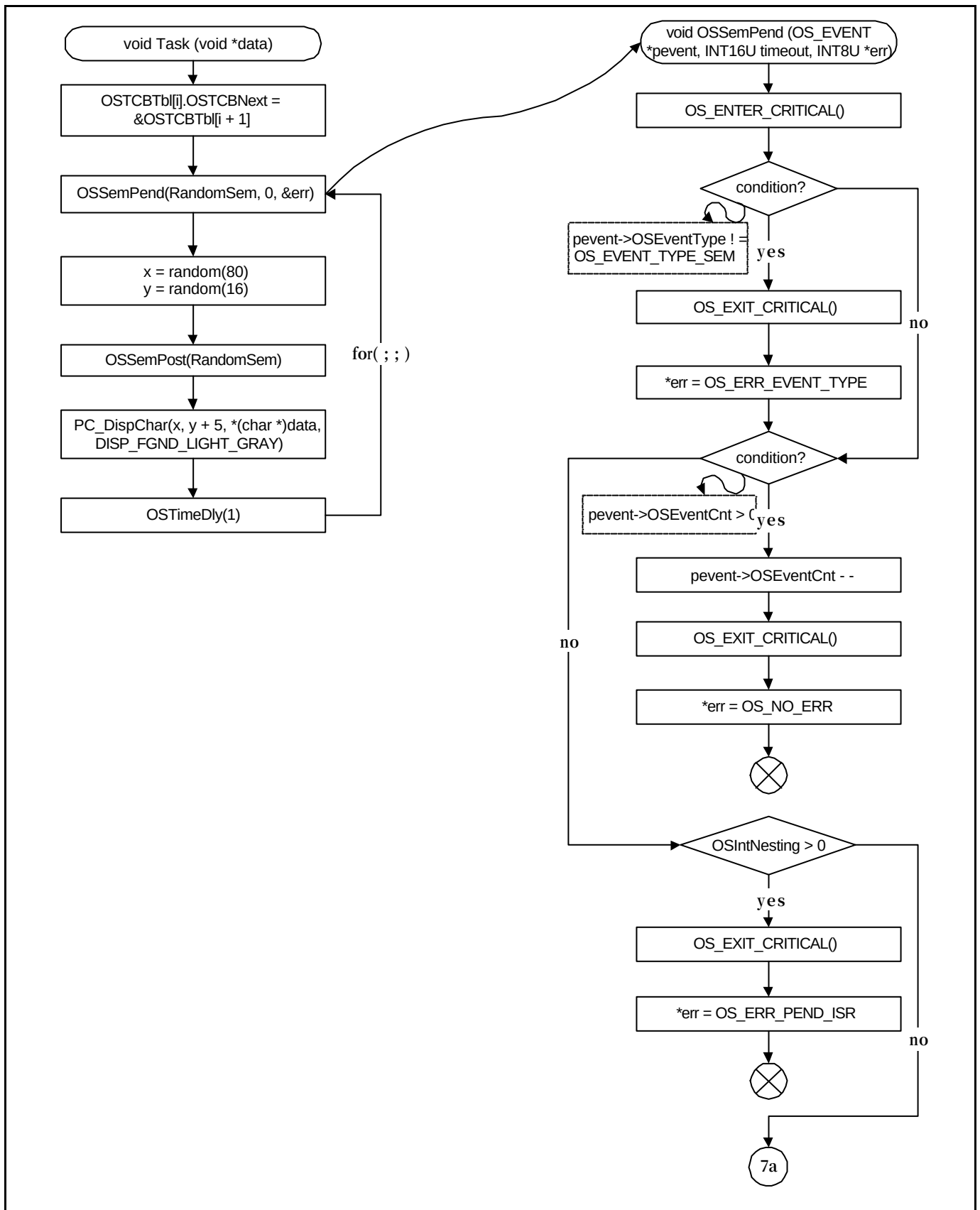
이전까지 모든 task들이 생성되었고 초기화 된 후에, 마지막으로 이런 task들을 멀티 테스킹 시켜주는 부분이다. 이 함수는 ex1.c 파일내의 main() 함수 안에 포함 된 함수이지만 이 함수는 다시는 종료되지 않는다. 중간부분에서는 앞에서 설명한 것과 같이 현재 생성된 task중 가장 우선순위의 task를 처음 실행시켜주는 일을 한다. 지금 생성된 task는 OSIdleTask(), OSStatTask(), OSTaskStart() task 이렇게 총 3개가 있으며, 이중 가장 높은 Task인 TaskStart() 가 실행될 것이다. 다음은 그 순차적인 과정을 그림으로 나타낸 것이다.

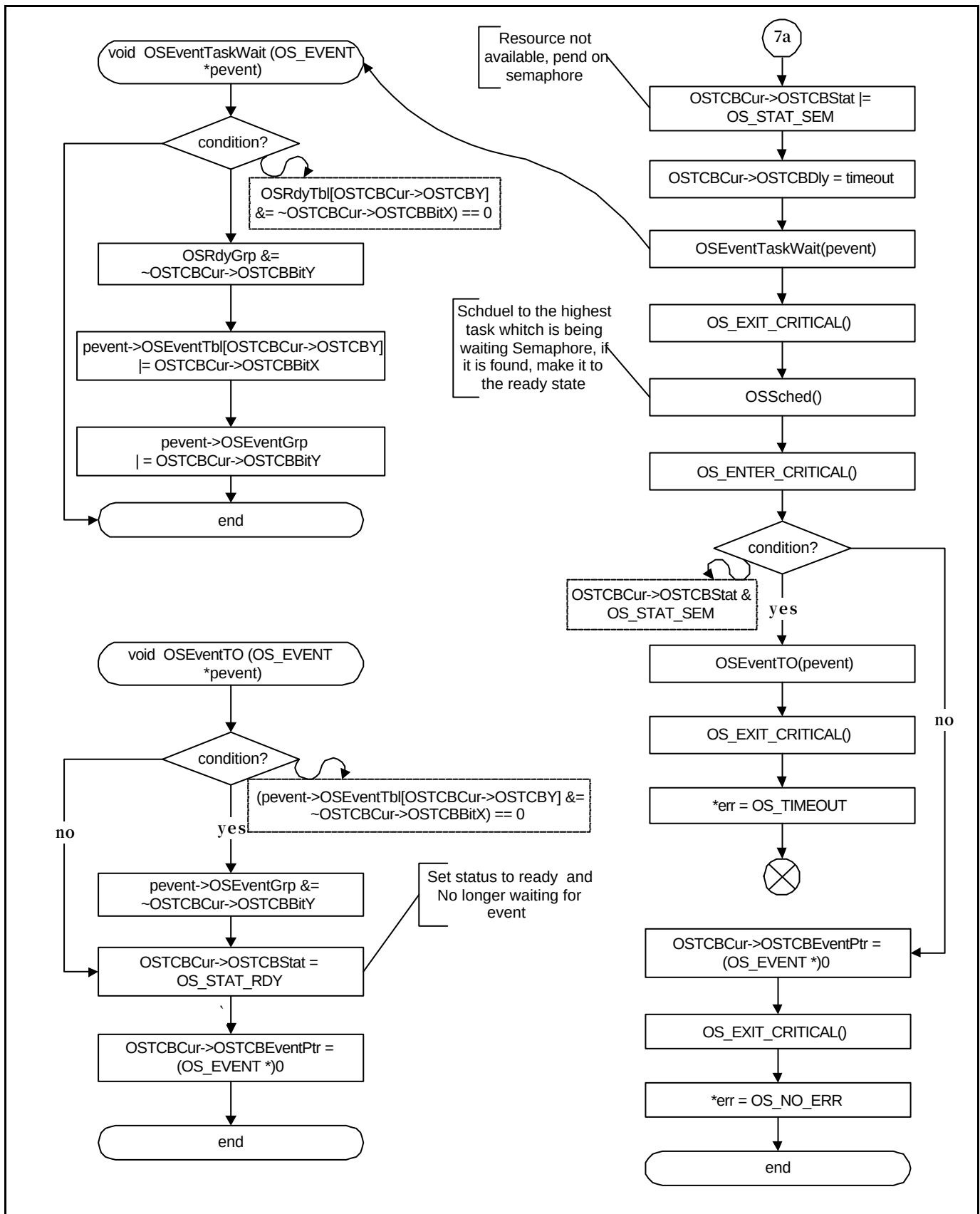
OSStart() 함수에 의해 현재 생성된 Ready Table 에 있는 task중 가장 높은 우선순위인 OSTaskStart()가 처음 실행된다. 이 task가 진행되던 도중 OSTimeDly() 를 만나고 이 함수에 의해 OSTaskStart는 Ready Table에서 제거된 후 2tick 동안 실행이 중단된다. 이후 스케줄링에 의해 그 다음 우선순위의 task인 OSTask Stat()가 실행되게 되는데, 현재 OSStatRdy변수가 FALSE 상태이기 때문에 이 함수 역시 OSTimeDly()에 의해 2초간 실행이 중단된다.

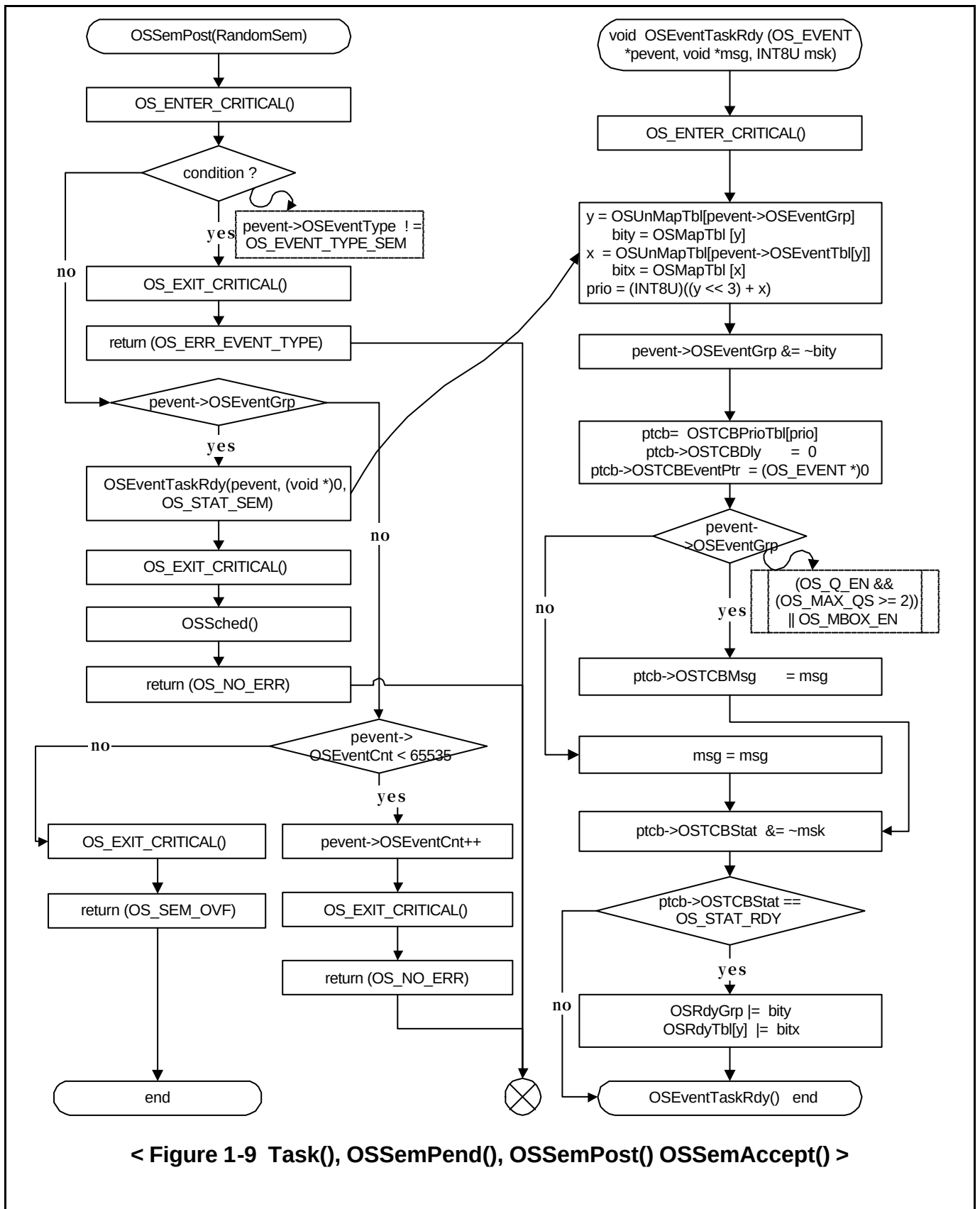
또 다시 스케줄링이 일어나 이번엔 OSTaskIdle()가 실행되지만 OSTaskStart()가 2tick이 지난 후 다시 Ready Table에 추가되면 이 task에 의해 실행이 다시 선점(preemptive)된다. 다시 실행되기 시작한 OS TaskStart()는 OSIdleCtr변수를 0으로 초기화한 후 다시 1 초간 Delay되고, OSTaskIdle() 이 1초간 아무런 방해 없이 OSIdleCtr를 계속 증가시켜 OSIdleCtr에 저장한다. 이 값은, 1초 후 다시 Reday 상태로 바뀌는 OSTaskStart()에 의해 OSIdleCtrMax 값으로 저장되고 난 후 나머지 코드들(Task생성 등..) 실행한다.

다음은 OSTaskStart() 함수가 생성하는 Task() 함수 에 대한 플로우 차트이다.









< Figure 1-9 Task(), OS_SemPend(), OS_SemPost() OS_SemAccept() >

11. void Task (void *data)

OSTaskStart() 함수에 의해 N_TASKS개의(10) 동일한 Task가 생성된다. 예제 1번에서는 이렇게 생성된 Task들이 세마포어를 이용하여 서로 동기화를 하게 되는데, 실행도중 OSSemPend(RandomSem, 0, &err) 세마포어가(자원) 할당 가능한지 살피고, 가능하지 않다면 수면상태가 된다. 만약 가능하다면 이 세마포어를 얻게 되고, 자신의 일을 수행한다. (이 테스트에서는 x, y에 각각 랜덤한 수를 만들어서 저장시킨 후 화면에 출력하는 일을 한다) 자신의 일을 마친 후, 할당 받은 세마포어를 OSSemPost()함수를 이용하여, 이 세마포어를 기다리는 다른 Task에 전달한다.

12, void OSSemPend (*pevent, timeout, *err)

세마포어를 기다리는 함수이다. 이벤트 카운터의 값이 양수이면 자원이 사용 가능하다는 의미이다. 이 경우 현재 EventCnt 변수를 하나 감소시킨 후 OS_NO_ERR 값을 리턴한다. 카운터 값이 음수라면 현재 자원이 유용하지 못하다는 것을 의미하고, 이 경우 현재의 Task가 세마포어를 기다리고 있다는 표시를 해둔 후(OSTCBStat=OS_STAT_SEM), 현재 Task가 얼마 동안 세마포어를 기다릴지에 대한 timeout 값을 설정한 후, OSEventTaskWait()함수를 이용해서 수면상태로 둔다.

OSEventTaskWait()

현재 Ready 상태에 있는 Task를 삭제하고, Event Table에 추가시켜 Wait 상태에 있도록 한다. 아래의 코드는 각각 Ready 상태에서 Task를 제거하는 코드이다. (OSRdyGrp &= ~OSTCBCur->OSTCBBitY)

하나의 Task Ready Table에서 삭제되었기 때문에, 여기서 다시 한번 스케줄링을 수행한다. 그렇게 해서 바뀐 Task가 역시 세마포어를 기다리는 중이라면, 바로 타임아웃이 적용되어 OSEventTo()함수에 의해 Event Table에서 이 Task를 제거한 후, 즉시 Ready 상태로 바뀌준다.

13. INT8U OSSemPost (OS_EVENT *pevent)

Task에게 세마포어를 전달하는 함수이다. 세마포어를 기다리는 Task가 있는지 확인하고, 이를 기다리는 Task가 있다면(OSEventGrp 변수체크), 이 중 가장 높은 우선순위를 가지는 Task를 OSEventTaskRdy() 함수를 이용하여, Wait에서 Ready상태로 바꾸어 준다. 이때 다시 한번 스케줄링을 수행해서 현재 바뀐 Task가

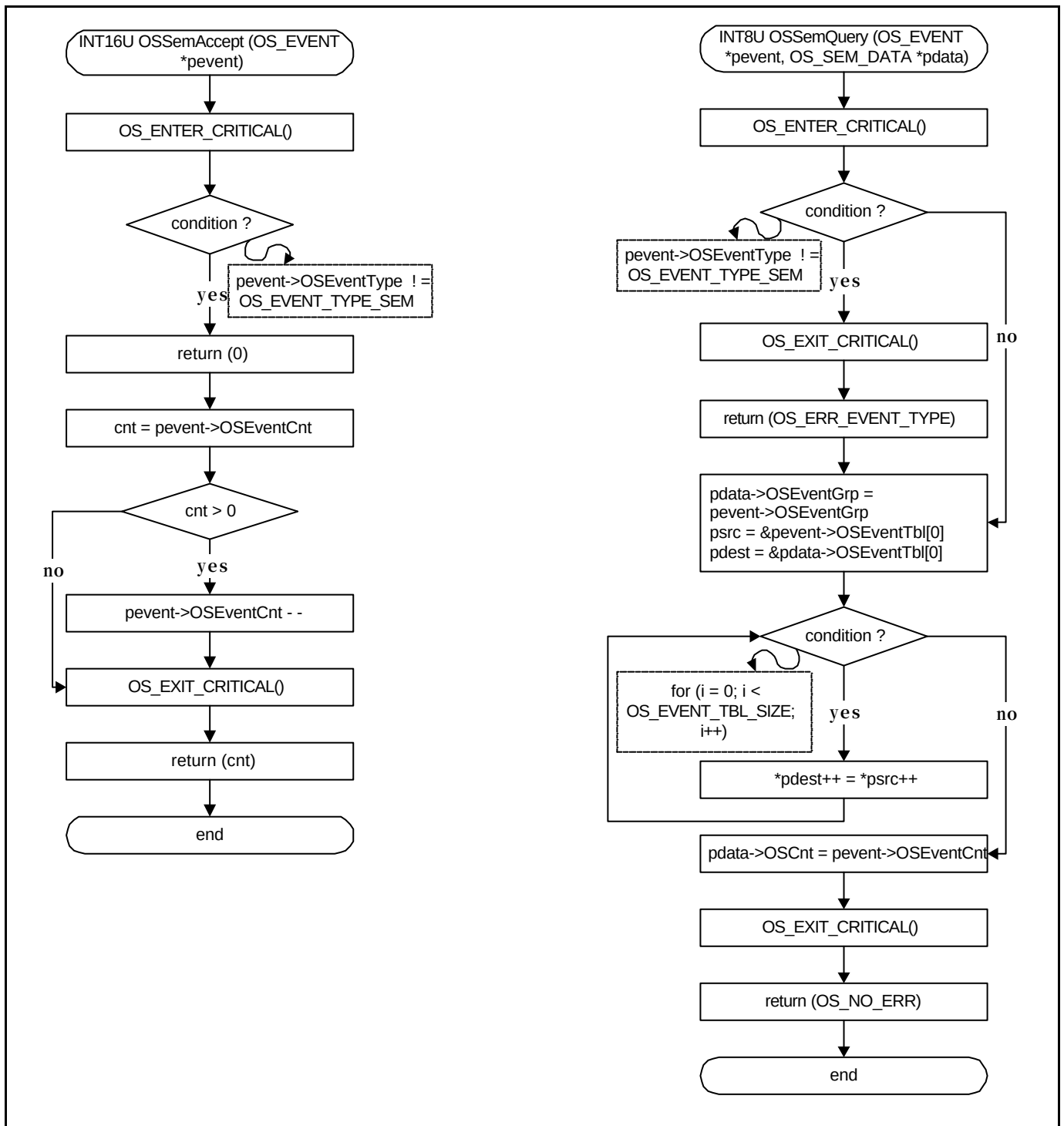
상태에서 가장 높은 우선순위를 가지는지 확인 한 후 Context switching 을 시도한다. 만약 그렇다면 이 Task를 세마포어를 얻게 되고 실행을 계속한다. 실행도중 발생하는 에러 값을 리턴한다.

14. INT16U OSSemAccept (OS_EVENT *pevent)

Task를 Wait 상태로 두지 않고 세마포어를 얻는 함수이다. 현재 세마포어가 사용가능 한지, (OSEventCnt 가 양수인지) 체크한 후, 그 값을 리턴 한다.

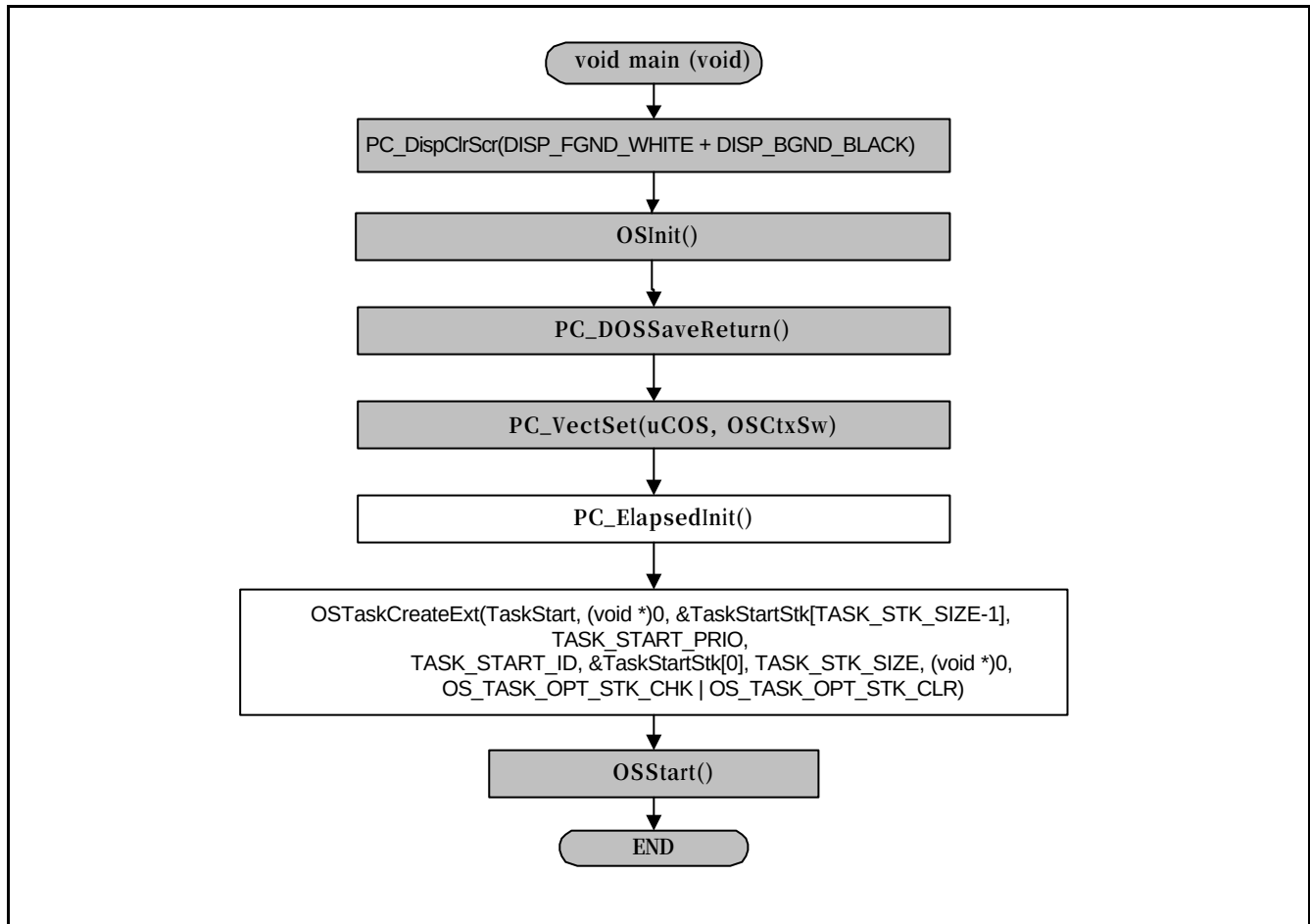
15. OSSemQuery()

현재 EventTable의 변수들을 그대로 캡춰한다.



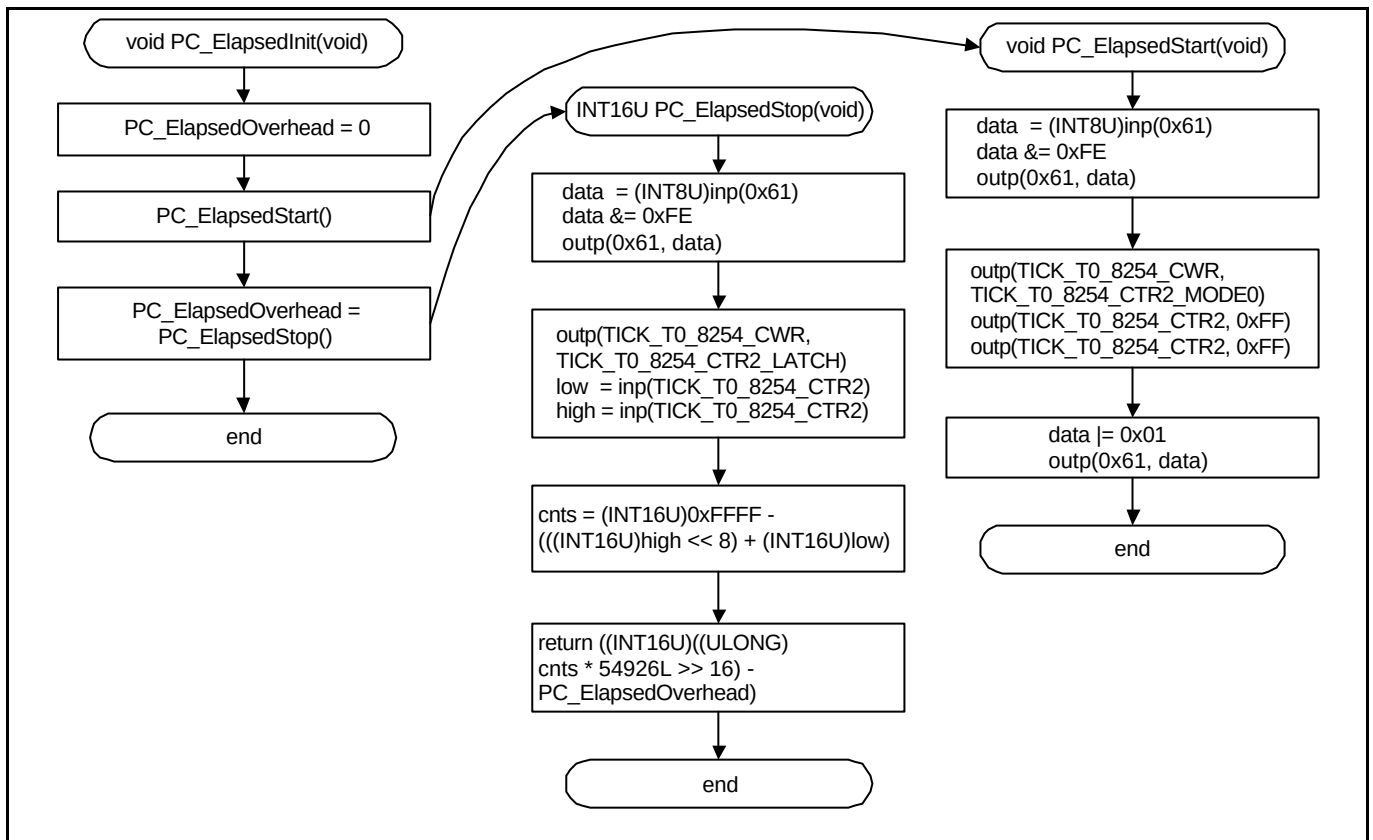
< Figure 1-10 OS_SemAccept(), OS_SemQuery() >

3 Program Flowchart (EX2l.exe)



< Figure 3-1 main() >

위의 순서도에서 흰색으로 표시된 부분이 예제 1과 비교한 예제 2번의 다른 부분이다. 예제 1에서는 이벤트 처리로 세마포어를 사용했지만 예제 2에서는 Message Mailbox를 사용한다. 그리고 OSTaskCreatExt() 함수로 태스크를 생성시키는 등의 몇 가지 다른 점이 있다. 중복되는 부분에 대한 설명을 최대한 피하기 위해 겹치는 순서도에 대해서는 회색으로 표시했다.



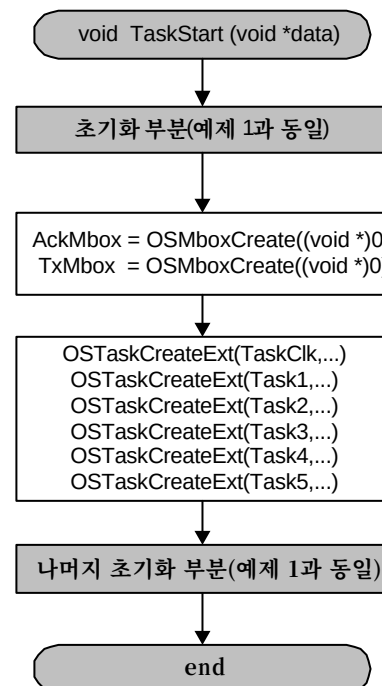
< Figure 3-2 PC_ElapsedInit(), TaskStart() >

1. PC_ElapsedInit()

PC_ElapsedStart() 함수와 PC_ElapsedStop() 함수가 쌍을 이뤄서 마치 스톱워치와 같은 기능을 한다. PC_ElapsedInit()는 두 함수 사이에 걸리는 Over Head값을 계산해서 실제 이 두 함수를 사용할 때 정확성을 높이기 위한 초기화 함수이다. PC_ElapsedStop() 함수는 ...Start() 함수가 실행된 이후 걸린 시간을 ms단위로 계산하여 리턴한다.

2. OSTaskCreatExt(TaskStart, ..)

TaskStart() 라는 Task를 생성한다. 이번 예제에서는 각각 다른 종류의 테스크 TaskClk, Task1~5까지의 Task가 더 생성된다는 점이다. 이 함수에서 테스크 생성 전 OSMboxCreate((void *)0) 함수를 이용하여 2개의 (OS_EVENT *) 형 포인터를 각각 AckMbox과 TxMbox로 리턴한다. 오른쪽은 그 순서도를 그린 것이다.



3. Task의 종류

예제 2에서는 TaskIdle(), TaskStat(), TaskStart() 들을 제외한 총 6개의 Task 들이 생성되며, 그 종류와 기능은 아래와 같다. *오른쪽은 우선순위 임

TaskStart	10
TaskClk	11
Task1	12
Task2	13
Task3	14
Task4	15
Task5	16

3-1. void TaskClk (void *data), <우선순위=11>

1초에 한번씩 실행되며 화면에 현재의 시간을 표시한다. 리턴 값은 없다.

3-2. void Task1 (void *pdata), <우선순위=12>

100ms 마다 한번씩 실행되어, 스택을 체크하고 거기에 걸린 시간을 재서 화면에 표시한다. OSTaskStkChk() 를 사용하며 현재 스택의 비어있는 크기를 체크한다.

INT8U OSTaskStkChk (prio, OS_STK_DATA *pdata)

위의 표에 있는 7개 테스크들의 비어있는 스택 값을 계산하고 그 스택 값을 리턴 한다. 다음 페이지는 이 함수에 대한 순서도이다

3-3. void Task2 (void *data), <우선순위=13>

10tick을 간격으로 화면에 |, /, -, \ 를 연속 출력한다.

3-4. void Task3 (void *data), <우선순위=14>

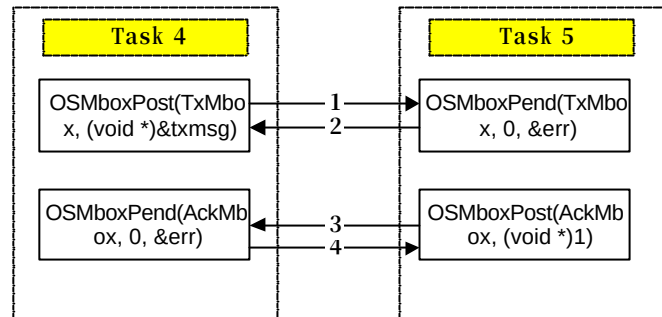
500바이트의 스택을 사용하고, 20tick을 주기로 Task 2 과 같은 문자를 연속 출력한다.

3-5. void Task4 (void *data), <우선순위=15>

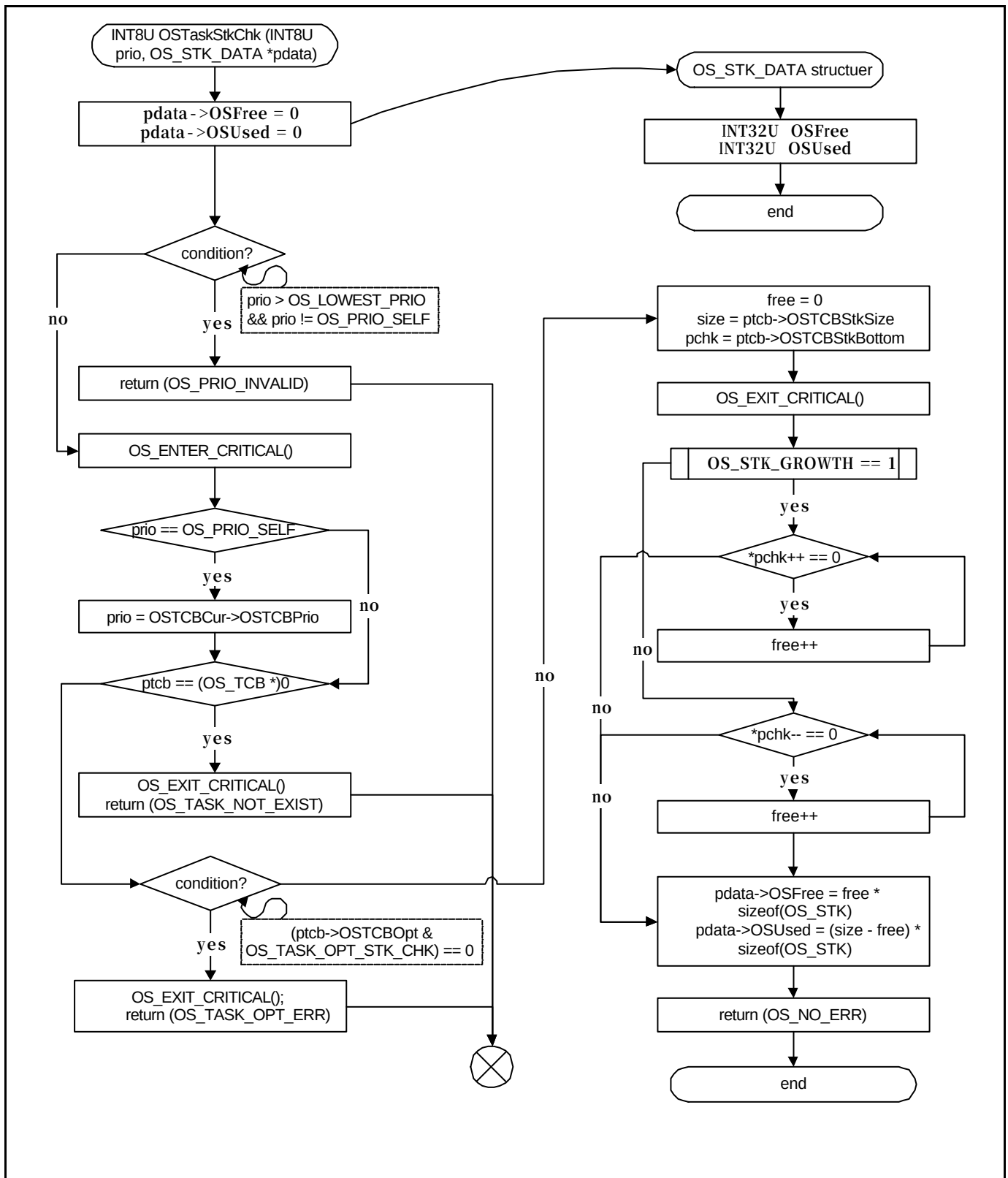
이 테스크는 5번 테스크에 메시지를 보내고, 이 문자는 5번 테스크에 의해 화면에 표시된다. 이 문자를 5번 테스크에 보낸 후 Ack신호를 기다린다.

3-6. void Task5 (void *data), <우선순위=16>

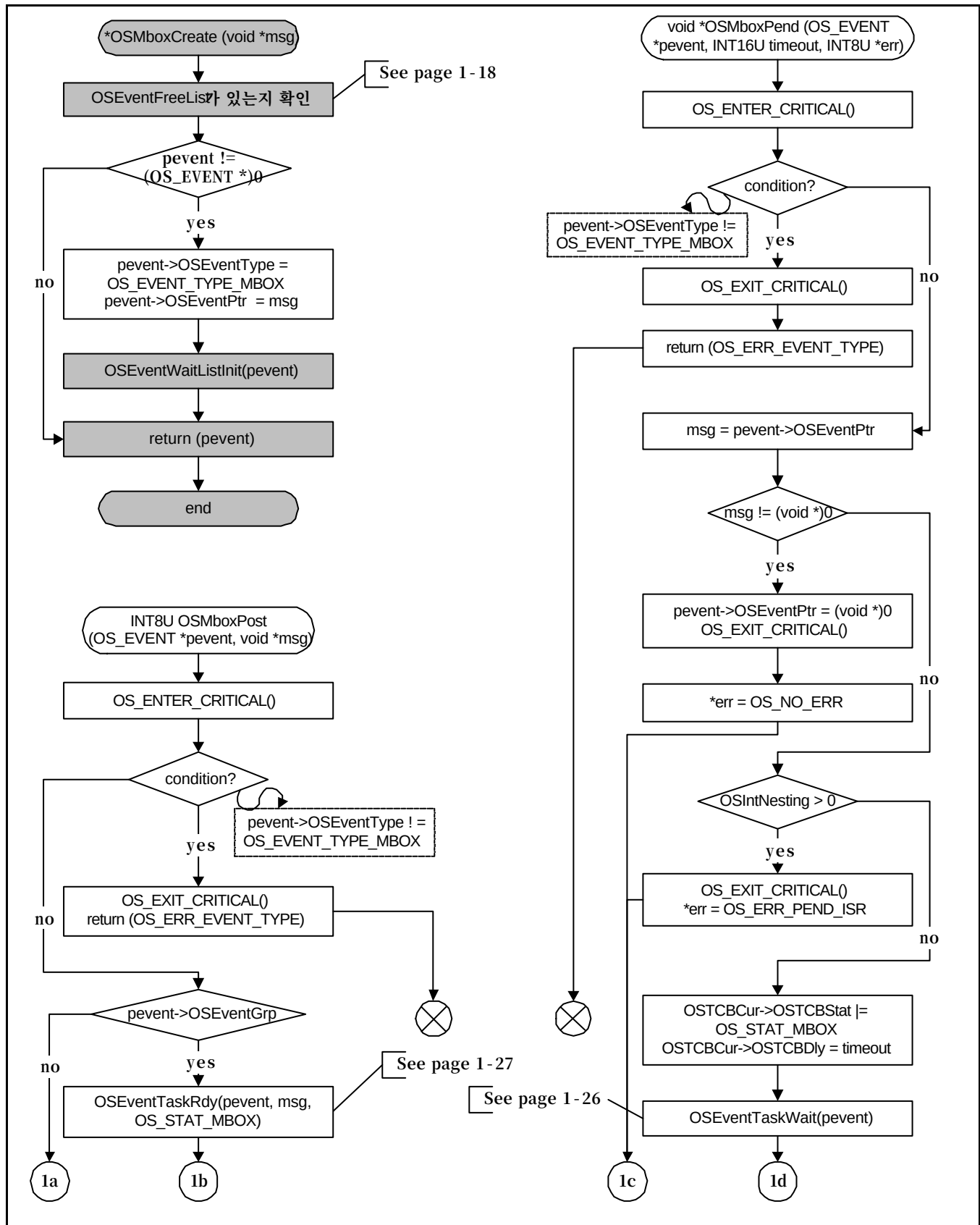
테스크 4 가 보낸 메시지를 받아들이고, 1초간 기다린 후 이에 대한 Ack 신호를 보낸다. 아래 그림은 두 테스크 간의 동기화 과정을 그림으로 나타낸 것이다.

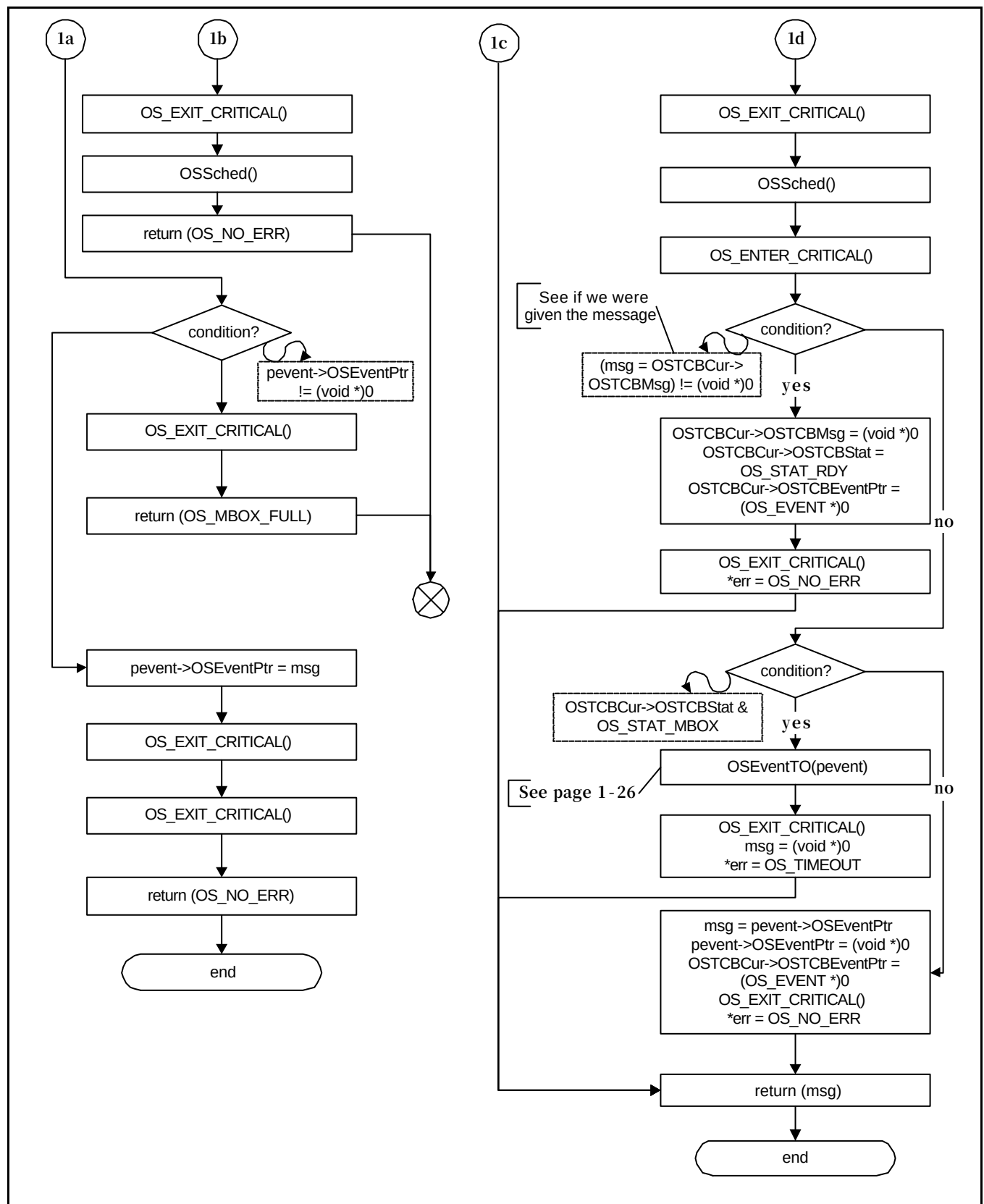


< Task Sync >



< Figure 3-3 OSTaskStkChk() >





< Figure 3-4 OSMboxCreat(), OSMboxPend(), OSMboxPost >

4. *OSMboxCreate (void *msg)

태스크간의 동기화를 맞추기 위해 Message Mailbox를 사용한다. 함수 자체는 앞에서 언급되었던 세마포어의 생성과정과 매우 비슷하며, 단지 OSEventType을 OS_EVENT_TYPE_MBOX 지정한다는 등의 미묘한 차이점이 있다. 하나의 EventList가 생성되면 그에 대한 포인터를 리턴한다.

5. OSMboxPost (*pevent, *msg)

메일 박스에 메시지를 보내는 함수이다. 현재 Event Table에 메시지를 기다리는 태스크가 있다면(pevent->OSEventGrp), OSEventTaskRdy() 함수를 이용하여, 그 중 가장 높은 우선순위의 태스크에게 이 메시지를 전달한다. (ptcb->OSTCBMsg = msg) 만약 메시지를 기다리는 태스크는 없는데 메일 박스는 차있는 상태라면 OS_MBOX_FULL 에러 메시지를 리턴 하며, 기다리는 태스크도 없고 아직 EventTable에 아무도 메시지를 가져다 놓지 않았다면(즉, 비어있다면) 이 메시지를 나중에 이를 필요로 할 어떤 태스크를 위해 Mailbox에 저장해 두고 함수는 종료된다.

6. *OSMboxPend (*pevent, timeout, *err)

메일 박스에 의해 전해지는 메시지를 기다리는 함수이다. 단순히 현재의 이벤트 포인터가 가리키는 구조체에서 OSEventPtr값을 msg 포인터로 받아오는 것이 궁극적인 이 함수의 기능이지만 여기에는 세마포어와 같이 몇 가지 예외 사항이 발생한다. 만약 받아온 msg 포인터가 비어있는 널 포인터라면(이 함수가 ISR로부터 호출된 것인지 확인한 후 맞다면 에러 메시지를 출력하고 종료) 현재 태스크가 원하는 메시지가 이 Mailbox에 들어있지 않는것을 의미하므로 이 태스크의 상태를 OS_STAT_MBOX로 바꾸고 (->pending on mailbox state), OSEventTaskWait() 함수를 이용해 이미 지정된 Delay값 동안 태스크는 메시지를 기다리는 Ready 상태로 바뀐다.

여기서 다시 한번 스케줄링을 하여 혹시 실행을 기다리는 더 높은 우선순위의 태스크를 확인하고, 만약 이때 다른 태스크가 선택된다면 코드는 여기서 실행을 중단하고 현재 태스크는 mailbox를 기다리는 Waiting 상태가 된다.

다시 태스크의 실행이 재시작 되는 경우는 OSMbox Post() 함수에 의해 메시지를 받았거나 TimeOut 되었거나 두 가지 경우 중 하나일 것이다. 다시 실행이 시작되면 Waiting 하는 동안 메시지를 얻었나 확인을 하고, (OSTCBMsg 변수확인) 얻었다면 더 이상 이 태스크

는 메시지를 얻고 Ready상태로 돌아간다. 하지만 아직 메시지를 얻지 못했다면 TimeOut 에러를 리턴 하고 종료된다. 그리고 마지막으로 아직 생각해 보지 않은 경우가 한 가지 더 있다. 바로 현재의 태스크에 전해진 메시지가 앞의 경우처럼 OSTCBMsg로 직접 태스크에 전해지지 않고, MailBox에만 들어 있을 경우이다. (pevent->OSEventPtr 변수확인) 이런 상황을 고려 한 것이 마지막 else 문이다

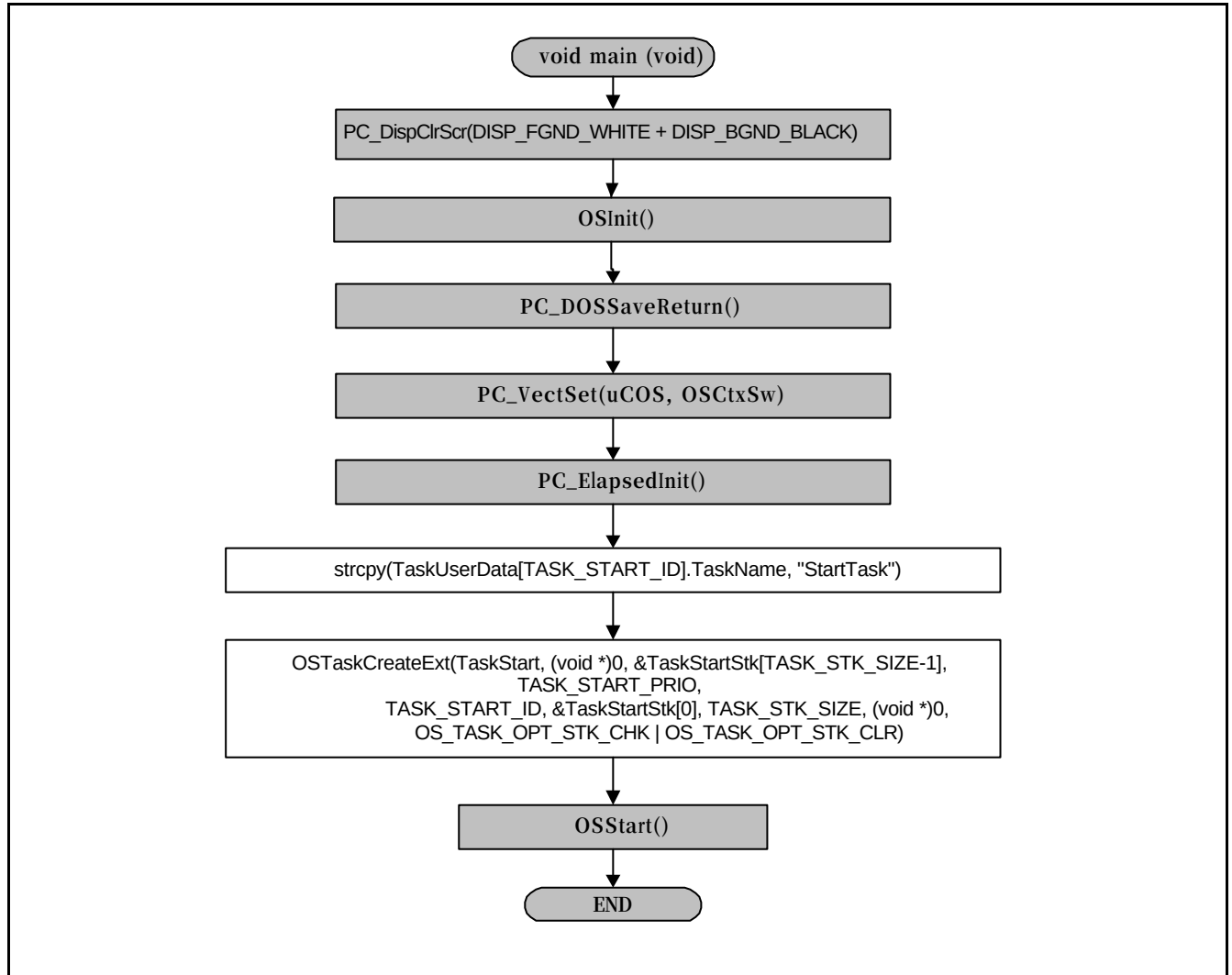
7. void *OSMboxAccept (*pevent)

세마포어에서와 같이 메일박스의 메시지가 사용가능한지를 체크한 후, 그렇지 않다고 하더라도 호출한 함수를 suspend시키지 않는다. 즉, 메시지가 있으면 그 값을 리턴 하지만, 없다고 하더라도 널 값을 리턴한다.

8. OSMboxQuery (*pevent, *pdata)

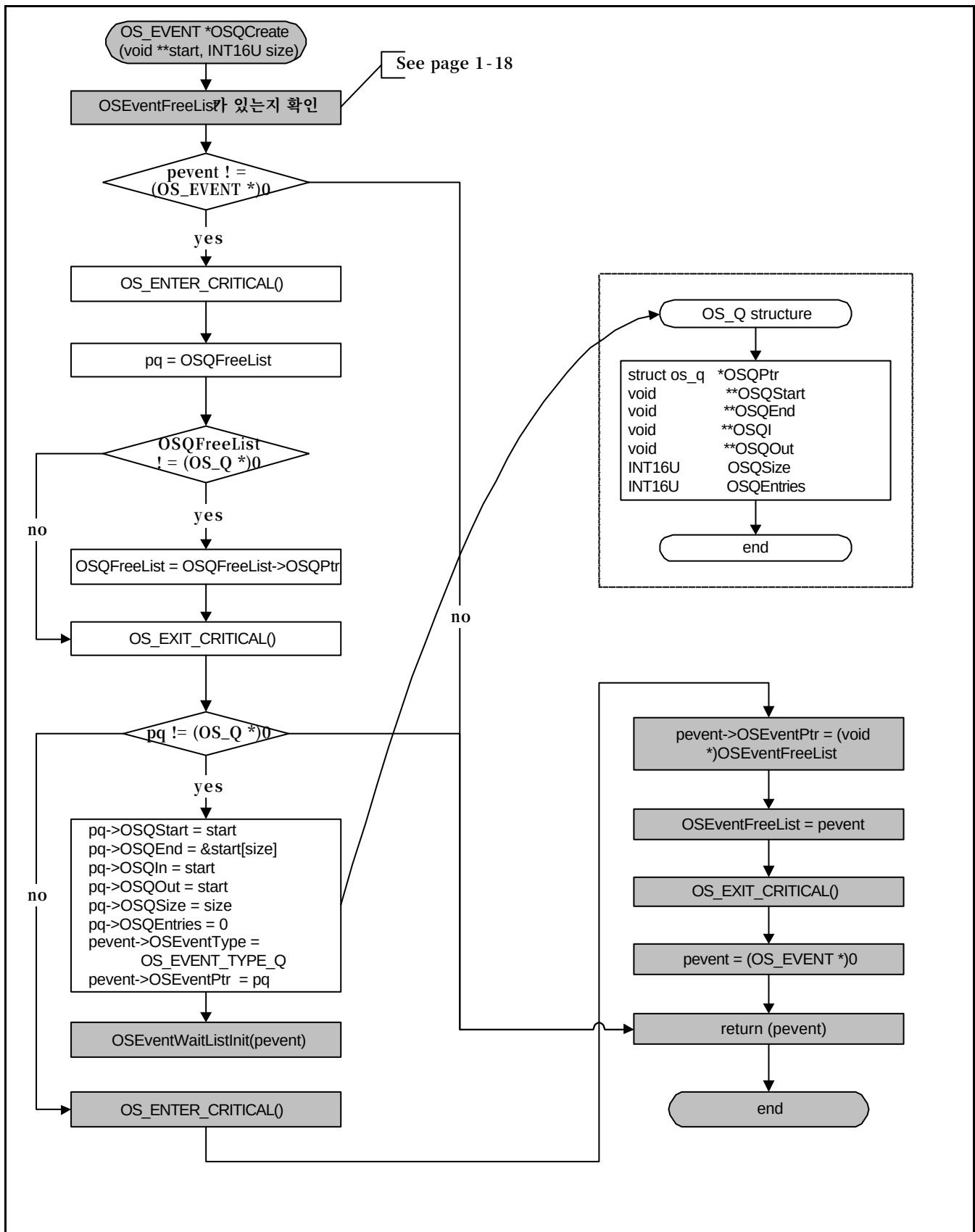
현재 *pevent에서 메시지를 기다리며 waiting 하고 있는 태스크들을 사진을 찍듯이 *pdata에 저장한다.

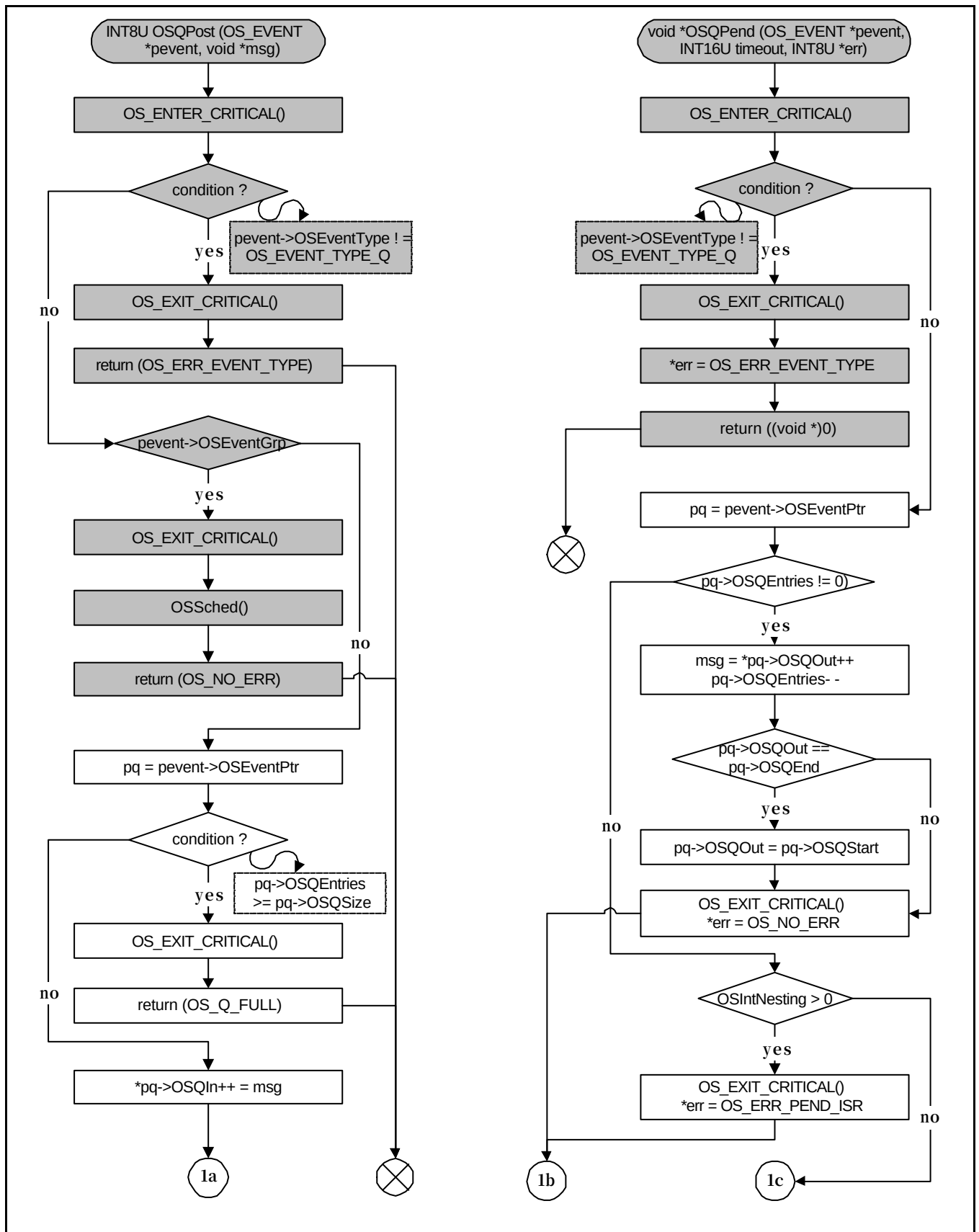
4 Program Flowchart (EX3l.exe)

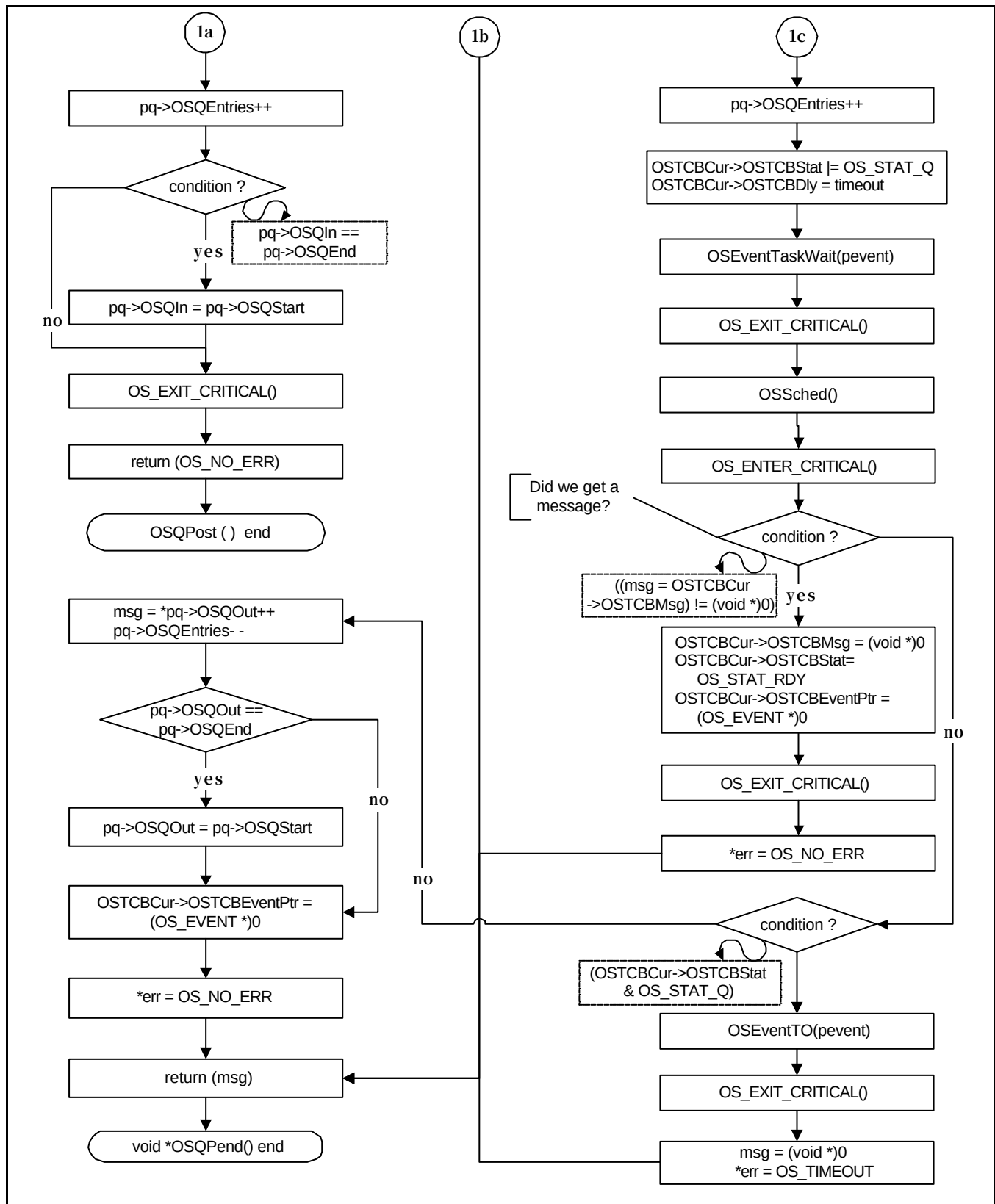


< Figure 4-1 main() >

역시 예제 2와 중복되는 부분은 회색으로 표시하였다. 좀 다른 부분은 처음 태스크를 생성하기 바로 전에 Task name, execution time 등의 user data를 TASK_USER_DATA 구조체에 저장 한다는 점이다. 다음 나오는 순서도는 OSTaskCraeatExt() 함수에 의해 생성되는 TaskStart() 태스크의 OSQCreate() 함수에 대한 부분이다. 그 이전의 다른 부분은 지금까지 보아온 다른 예제와 거의 동일하므로 생략한다. 큐가 생성된 후 OS가 사용할 나머지 태스크들이 계속 만들어진다.(TaskClk, Task1~5)



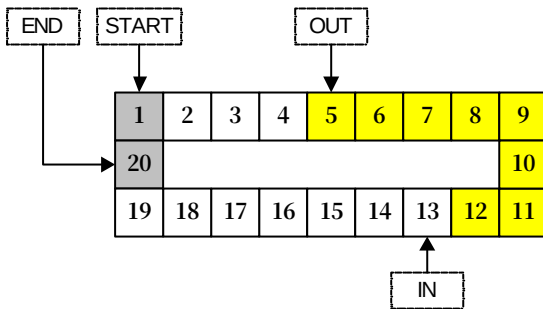




< Figure 4-2 OSQCreat(), OSQPost(), OSQPend() >

1. OSQCreat()

태스크들이 서로 비 동기적으로 공유하는 자원을 효율적으로 사용하기 위해 큐(Queue)를 사용한다. 처음 OSQInit() 함수에 의해 초기화가 이루어질 때 (pg 1-6 참고) 총 5개의 OSQList가 만들어지며, 여기에서 사용되는 큐는 void형 포인터의 배열로서 20개의 요소를 가지는 원형 리스트이다(아래 그림)



위에서 노란색이 현재 메시지가 저장 되어 있는 부분이며, 5번이 다음 메시지를 읽을 부분, 12번은 메시지가 다음에 기록될 부분을 각각 표시한다.

2. OSQPost()

메시지를 보낼 Event Table에 태스크가 존재하면 이 태스크에 메시지를 전달한다. 이 메시지를 기다리는 아무런 태스크도 아직 없다면 OSEventPtr이 가리키는 큐리스트가 기록이 가능한지 확인 후 메시지를 저장해 놓는다. 만약 큐가 가득 차 있다면 에러 메시지를 리턴한다. 메시지를 저장한 후 OSQIn 변수를 하나 증가시킨다.

3. OSQPent()

다만 메시지가 저장되고 불러지는 장소가 큐라는 리스트로 확장 되었을 뿐 거의 모든 기능이 MailBox와 비슷하다. 메시지를 사용했다면 OSQOut 변수를 하나 증가시킨다. 다른 중복되는 설명은 생략한다.

4. Task의 종류

4-1. Task1()~Task3()

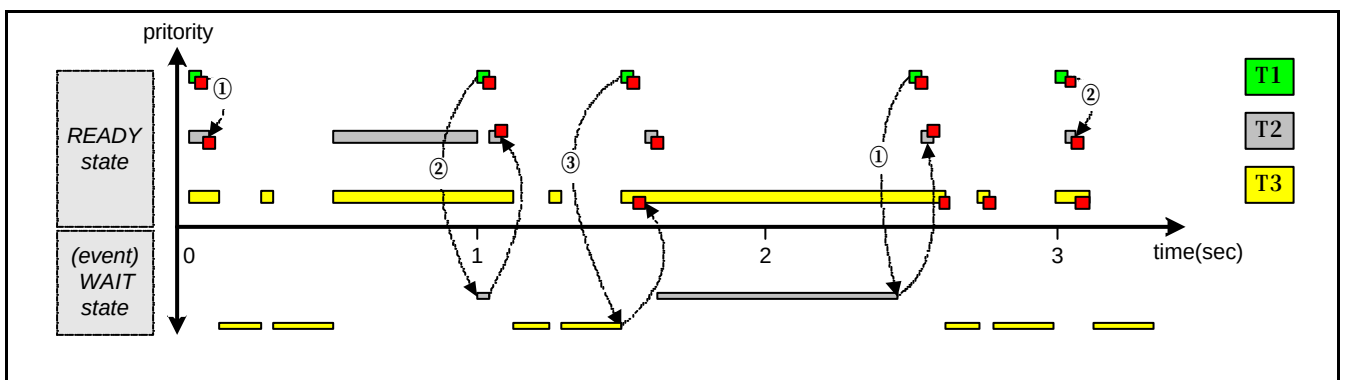
T1은(태스크1번) 숫자 1,2,3의 메시지를 일정한 시간 텀을 가지고 계속 발생시키며, T2, T3는 이 메시지를 받아 화면에 출력 한다. 아래는 그 과정을 그림으로 그려보았다. 주의해서 보아야 할 부분은 '준비상태'와와 '실행상태'는 별개의 문제라는 점이다. 밑에 빨간 박스로 표시되어 있는 부분이 실제 태스크 들이 '실행되는 순간'을 의미한다. 처음 0초인 순간 모든 태스크가 준비상태에 있고 처음 실행되는 T1이 메시지(숫자1)을 큐에 저장하면 다음 스케줄링이 되면서 T2가 이를 받아 화면에 출력 한 후, Delay time 을 갖는다. 이때 T2는 더 이상 메시지를 기다릴 필요가 없 으므로 Wait 상태로 가지는 않는다. 다음 T3가 실행 되지만 메시지는 남아 있지 않으므로 TimeOut Delay (250ms) 동안 메시지를 기다린다. 비슷한 방식으로 두 번째 메시지(숫자2) 역시 T3는 T2에 밀려 사용하지 못하게 되지만 T2가 메시지(숫자2)를 받고 나서 Delay 상태에 있는 동안 계속 메시지를 체크하던 T3는 T1이 세 번째로 만들어내는 메시지(숫자3)을 드디어 사용할 수 있게 된다. 잠시 후 Delay time 동안 쉬던 T2는 깨어나 메시지를 찾아보지만 이미 T3가 가져간 후 이기에 다시 대기하며 다음 메시지(숫자1)를 기다 린다.

4-2 Task4()

10ms에 한번씩 실행된다. 이 태스크는 메일박스를 만들어 사용하는데, 단순히 혼자서 메시지를 저장하고 이를 불러 다 사용하는 일만 수행한다.

4-3 Task5()

1초에 한번씩 실행되지만 하는 일은 없다.



맺음말

아직까지도(?) uC에 관한 글이 많이 올라오는 거 같아 예전에 공부하면서 정리해 둔 세미나 자료를 올립니다.

전 운영체제 수업도 아직 들어 본 적이 없는 3학년 학부생입니다. uC책도 너무 비싸서 차마 못 사고 무작정 소스와 인터넷에서 찾은 자료들을 이용해서 분석해 본 수준입니다. 분명 틀리거나 부족한 부분도 많으리라 생각합니다.

그저 한번쯤 전체적인 구성이 이러하구나 하고 살펴보시는데 조금이나마 도움이 되었으면 하는 마음입니다.

날씨가 춥네요. 감기 조심하세요.

2001 겨울